

Unidad 1 - Ingeniería de software en contexto

Introducción a la ingeniería de software. ¿Qué es?

Software

El software es un conjunto de programas en conjunto con **toda** la documentación requerida para definir, desarrollar y mantener los programas ejecutables que se entregan al cliente (archivos de configuración, documentos p/ el usuario, documentos sobre la estructura del sistema), como así también las herramientas utilizadas para la construcción de este.

Ingeniería de Software

Es una disciplina que se encarga de todos los aspectos de la producción del software, desde la primera etapa de especificación del sistema hasta el mantenimiento del sistema después de que se pone en operación.

Ingeniería de Software → Parmas [1987] definió a la ingeniería en software como “multi-person construction of multi-version software”

Esta ingeniería es importante ya que:

- Con mayor frecuencia se requiere producir de manera rápida y económica sistemas confiables.
- Resulta más barato a largo plazo utilizar métodos y técnicas de ingeniería de software para los sistemas de software.
- La mayoría de los costos consisten en cambiar el software después de que se pone en operación.

Además, la ingeniería de software se apoya en varias realidades, entre ellas:

- Se debe entender el problema y las necesidades del cliente antes de proponer una solución.
- El diseño es una actividad crucial en el desarrollo de un software.
- Realizar un correcto diseño del software, trae consigo dos claros beneficios, calidad de software y facilidad de mantenimiento.

La ingeniería de software nace tras las **crisis del software**, en la cual existía (y existe) un conjunto de dificultades o errores ocurridos en la planificación, estimación de los costos, productividad y calidad de un software, debido, principalmente, a la baja eficacia que presentan una gran cantidad de empresas al momento de desarrollarlo.

Estado actual y antecedentes. La crisis del software.

El término crisis del software hace referencia a un conjunto de hechos relativos al software, planteados en la conferencia de OTAN en 1968 por Friedrich Bauer, quien recalco la dificultad para generar software libre de defectos, fácilmente comprensibles y que sean verificables. Sin embargo, este término fue utilizado anteriormente por Dijkstra en El Humilde Programador.

Causas

- La evolución de la tecnología del hardware mediante los circuitos integrados permitió la posibilidad del desarrollo de sistemas más grandes y complejos. El salto en el hardware no fue acompañado por un salto en el desarrollo del software.
- En combinación con lo anterior, la demanda creciente de estos sistemas complejos, la subestimación de la complejidad que supone el desarrollo de software, los cambios a los que tienen que ser sometidos los productos para ser adaptados a las necesidades del cliente y la falta de una disciplina que intervenga en todos los aspectos de la producción del software fueron las causas de esta crisis. 8

En la actualidad, muchos de los sistemas desarrollados fracasan debido a fallas en el software, que principalmente son consecuencia de dos factores:

- Demandas crecientes: la demanda de sistemas más grandes y complejos aumenta. Los usuarios necesitan que se construyan y distribuyan de manera rápida. La problemática aquí es que los métodos tradicionales no permiten lograr esta “espontaneidad”, por lo que se deben desarrollar y poner a prueba nuevas técnicas y métodos de gestión y desarrollo.
- Bajas expectativas: muchas de las compañías actuales de desarrollo de software no utilizan métodos de ingeniería de software en su trabajo diario. Por lo tanto, su software con frecuencia es más costoso y menos confiable de lo que debiera. La consecuencia, las empresas se conforman con que el software solo “funcione”, sin tener en cuenta el valor agregado que el software le debe aportar al negocio del usuario. Es necesaria una mejor educación y capacitación en ingeniería de software para solucionar este problema.

Disciplinas que conforman la ICW

Disciplinas Técnicas

- Toma de requerimientos
- Análisis de requerimiento
- Diseño de software
- Implementación de software
- Prueba de software
- Despliegue de un software
- Documentación de software
- Capacitaciones a usuarios

Disciplinas de Gestión

- Planificación de Proyecto
- Monitoreo y Control de Proyectos

Disciplinas de Soporte

- Gestión de Configuración de Software
- Aseguramiento de Calidad
- Métricas

Ejemplos de grandes proyectos de software fallidos y exitosos.

Software Exitoso

1. GNU/Linux: Es considerado un éxito rotundo por haber revolucionado la informática. Para septiembre de 2009, 446 de los 500 superordenadores más potentes del mundo ya utilizaban este sistema operativo. Además, es la base de los teléfonos con Android.
2. Apache: Es el servidor web por excelencia, alcanzando una cuota de mercado del 54% en 2010. Su versatilidad y su naturaleza de software libre le permitieron convertirse en el estándar de facto para servir páginas web.

Software Fallido

1. Ubuntu Phone: Se describe como un "fracaso espectacular". Su interfaz resultó casi imposible de usar para los usuarios, carecía de aplicaciones propias y la calidad de las llamadas telefónicas era absurdamente mala.
2. Steam Machine: Fue una consola de juegos basada en Linux desarrollada por Valve para atraer a los jugadores de PC al living de sus casas. Sin embargo, fracasó de tal manera que ni siquiera existe una explicación oficial clara, simplemente desapareció de los sitios oficiales

Ciclos de vida (Modelos de Proceso) y su influencia en la Administración de Proyectos de Software.

Ciclo de vida → una serie de pasos a través de los cuales un producto o un proyecto progresa en el tiempo.

Define:

- Elementos del proceso (actividades estructurales, productos del trabajo, tareas, etc.)
- El flujo del proceso (o flujo de trabajo), que especifica la relación y el orden de dichos elementos.

Además, establece los criterios que se utilizan para determinar cuándo se procede de una tarea a otra.

Los modelos de ciclos de vida especifican:

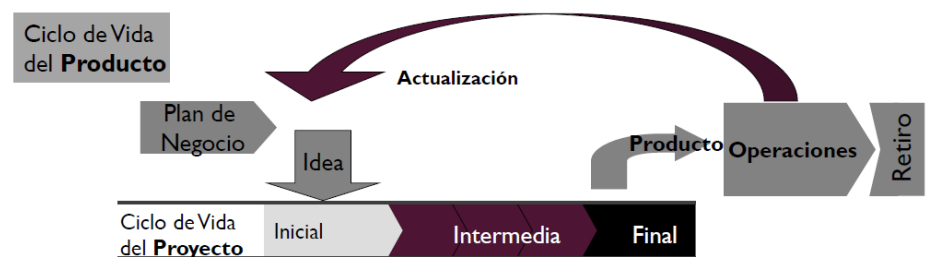
- Las fases del proceso (requerimientos, especificaciones, diseño, etc.)
- Orden en el cual se llevan a cabo.

Relación entre el ciclo de vida del proyecto y del producto

No existe un impacto entre los ciclos de vida de cada uno, sino que el ciclo de vida del producto siempre es mayor que el ciclo de vida del proyecto, ya que el ciclo de vida del proyecto dura lo que dura el desarrollo del software. En cambio, el ciclo de vida del producto dura hasta que el software se deje de utilizar.

Es posible que un producto tenga varios proyectos en su ciclo de vida, debido a, constantes cambios y/o actualizaciones que se vayan realizando.

Por lo que, dentro del ciclo de vida del producto, se pueden desarrollar varios ciclos de vida de proyectos.



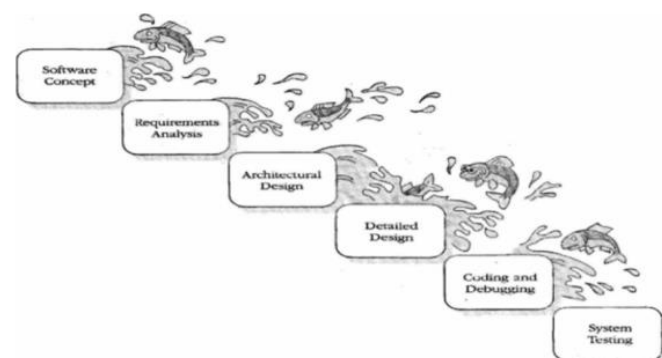
Clasificación de los ciclos de vida

Modelo Secuencial

Este modelo dispone de las actividades de forma lineal, es decir, que el proyecto progresa a través de una secuencia ordenada de pasos (fases) y una actividad no puede iniciar sin que la precedente haya sido finalizada. El avance entre las fases se da tras una revisión al final de cada una de estas para determinar si el software está listo para avanzar.

Se suele utilizar en aquellos sistemas donde los requerimientos se comprenden bien y son exhaustivos ya que éstos se congelan al finalizar la etapa de toma de requerimientos.

Generalmente, este tipo de modelos está dirigido por documentos, es decir, el trabajo principal del producto es la documentación del software entre las distintas fases.



Escenarios de proyectos donde elegirlo:

- El modelo en cascada puro ayuda a minimizar los gastos generales de planificación porque se puede hacer toda la planificación por adelantado. No proporciona resultados tangibles en el desarrollo de software hasta el final del ciclo de vida, pero la documentación que se genera a lo largo de este proporciona indicaciones significativas de progreso a lo largo del ciclo de vida.
- Es más sencillo utilizar un método de gestión común a lo largo de todo el proyecto, por lo que aún es común el uso de este modelo.
- El modelo en cascada puro funciona bien para productos en los que se tiene una definición clara y estable, y cuando se trabaja con metodologías técnicas claras.

Dificultades

- Los proyectos raramente siguen un flujo secuencial.
- El modelo secuencial exige que los requerimientos sean completamente explícitos desde un principio y esto en la realidad no suele suceder. No puede lidiar con la incertidumbre.
- El cliente obtiene una versión funcional del producto en etapas muy avanzadas del proyecto.
- Encontrar un defecto implica un gran rediseño en la solución, ya que está prácticamente todo hecho.
- Dependencia entre los equipos de trabajo. Un equipo no puede comenzar hasta que el otro no haya finalizado.
- El desarrollo está dirigido por un plan, y cada etapa genera documentación para realizar un monitoreo constante contra el plan, por lo que esa documentación puede llegar a ser muy burocrática y excesiva.

Modelo iterativo/incremental

Este modelo aplica sucesivas iteraciones en forma escalonada a medida que avanza el calendario de actividades.

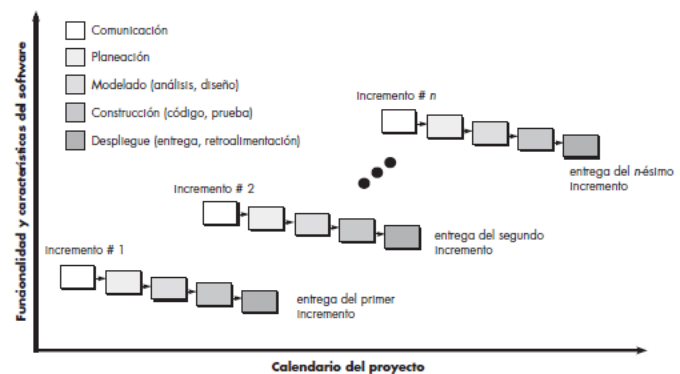
Cada iteración produce un incremento de software funcional potencialmente entregable.

Usualmente los primeros incrementos incluyen las funciones básicas/críticas que más requiere el cliente.

Este enfoque vincula las actividades de especificación, desarrollo y validación.

El sistema se desarrolla como una serie de versiones (incrementos) y cada una añade funcionalidades a la versión anterior.

Este ciclo de vida se utiliza en metodologías ágiles.



Ventajas frente al modelo secuencial

- Se reduce el costo de adaptar los requerimientos cambiantes del cliente y el retrabajo es menor ya que en cada incremento se obtiene una retroalimentación que permite adaptar el modelo.
- El cliente es parte del proceso de desarrollo. En el modelo secuencial, el cliente debía esperar hasta la última instancia para recibir el producto y no era capaz de juzgar el avance del producto a través de documentos de diseño de software.
- Es posible que la entrega se a más rápida la entrega e implantación de software útil al cliente, aún si no se ha incluido toda la funcionalidad.
- Muy útil para sistemas de requerimientos cambiantes.
- La especificación, desarrollo y validación están entrelazadas en lugar de separadas y aisladas.

Dificultades desde una perspectiva administrativa

- Se invisibiliza el proceso.
- Alto costo de documentar cada incremento.
- La estructura del software tiende a degradarse frente a una cantidad considerable de incrementos.

Modelo Recursivo

El modelo recursivo es utilizado para gestionar los riesgos del desarrollo de sistemas complejos a gran escala. Requiere de la intervención del cliente. El modelo en espiral es un modelo recursivo, que consta de 4 etapas recursivas:

1. Determinar objetivos.
2. Identificar y resolver los riesgos.
3. Desarrollar y probar.
4. Planificar la próxima iteración.

Para llevarlo adelante, se subdivide el proyecto en varios mini proyectos, intentando resolver en cada uno los riesgos más relevantes hasta que no quede ninguno.

Dicho en otras palabras, se inicia con algo en forma completa, como una subrutina que se llama a sí misma e inicia nuevamente. Se presenta un prototipo que va mejorando con cada vuelta. Lo positivo, es que se generan productos independientes de la implementación, que pueden ser reusables en sistemas de características similares.

Dificultades desde una perspectiva administrativa

- Puede ser más costoso en tiempo y dinero readaptarlos para reutilizarlos para diferentes proyectos.
- La tecnología puede ser obsoleta.
- Puede carecer de mantenimiento o documentación, lo cual dificulta mucho las tareas.
- Se genera una versión del producto recién al final.

Administración de proyectos dependiendo en ciclo de vida

La elección de un ciclo de vida afecta de forma directa a la administración que se debe realizar sobre el proyecto, ya que cada uno de los modelos de proceso poseen características y enfoques distintos.

Por ejemplo, si se realiza un análisis de riesgo y se determina que los requerimientos pueden variar mucho a lo largo del desarrollo, se debería elegir un ciclo de vida en el cual el cambio sea permisible, como los iterativos.

En este caso, la administración y la planificación del proyecto se ve afectada en la elección del ciclo de vida.

Procesos de Desarrollo Empíricos vs. Definidos.

Proceso de desarrollo de software

Conjunto estructurado de actividades, métodos, prácticas y transformaciones para desarrollar un sistema de software

Estas actividades varían dependiendo de la organización y el tipo de sistema que debe desarrollarse.

Debe ser explícitamente modelado si va a ser administrado.

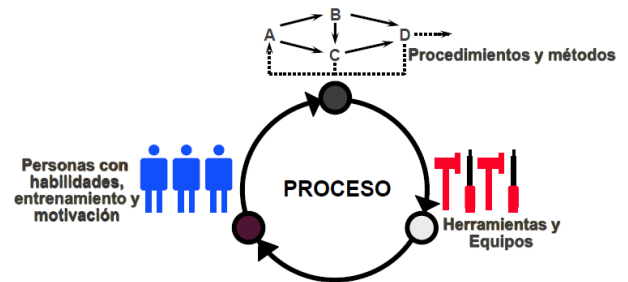
Las actividades del proceso de desarrollo son la especificación de requerimientos, el análisis, el diseño, la implementación y la prueba.

El orden y la relación entre las diferentes actividades está determinada por la elección del ciclo de vida.



Se tienen tres factores determinantes del proceso:

- **Procedimientos y Métodos:** Deben **adaptarse al contexto** específico del proyecto y estar debidamente documentados para garantizar la transparencia y visibilidad de la gestión.
- **Personas:** Es el factor más crítico, ya que el desarrollo es una **actividad "humano-intensiva"**. El éxito depende del esfuerzo, la capacitación y la motivación de los individuos para resolver la complejidad conceptual del sistema.
- **Herramientas y Equipos:** Actúan como soporte para transformar entradas en salidas. Se busca **automatizar tareas rutinarias** (mediante herramientas CASE o IDEs) para que el personal pueda enfocarse en actividades que requieren su capacidad intelectual.



Conclusión: No existe un **proceso ideal**. La elección depende del sistema: los **sistemas críticos** exigen procesos muy estructurados y definidos, mientras que los **sistemas empresariales** con requerimientos cambiantes son más efectivos bajo una **filosofía ágil** y flexible.

Consideraciones

1. Estas actividades varían dependiendo de la organización y el tipo de sistema que debe desarrollarse, pero deben incluir: productos, roles, responsabilidades y condiciones
2. El proceso debe ser explícitamente modelado si va a ser administrado.
3. Un proceso de desarrollo se puede aplicar a varios ciclos de vida. La elección del ciclo de vida depende de la estrategia y este debe permitir seleccionar los artefactos a producir, definir actividades y roles, y modelar conceptos.
4. Las personas utilizan herramientas y equipos para llevar a cabo los procedimientos que componen el proceso de desarrollo.

Actividades fundamentales

1. **Especificación de software:** clientes junto con ingenieros definen el software a producir y sus restricciones.
2. **Desarrollo:** diseño y programación del software.
3. **Validación:** verificación para asegurar que es lo que el cliente quiere.
4. **Evolución:** donde se modifica el software para reflejar los requerimientos cambiantes del cliente y mercado.

Procesos Definidos (Deterministas)

Se basan en la idea de que el desarrollo de software puede tratarse como una **línea de producción** industrial.

Son considerados deterministas: Ante la misma entrada, se debe obtener siempre la misma salida.

Objetivo: Lograr **repetibilidad y previsibilidad** en costos, tiempo y riesgos.

Gestión: El control se centra en seguir el plan y en la **documentación exhaustiva** del avance, a veces dándole más importancia a los papeles que al software funcional.

Limitación: En la realidad, el software es una actividad "humano-intensiva" y el contexto o el equipo siempre cambian el resultado final.



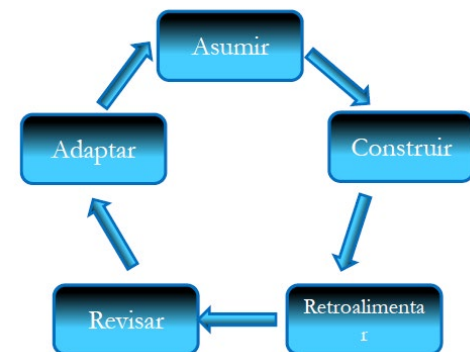
Procesos Empíricos (No Deterministas)

Se basan en la **experiencia sensible** y la adaptación constante al entorno.

Se sustentan en la **inspección** frecuente, la **adaptación** (retroalimentación) y la **transparencia** dentro del equipo.

La administración y el control es por medio de inspecciones frecuentes y adaptaciones para lograr buenas prácticas.

- **Ciclos cortos:** Utilizan ciclos de trabajo breves para ajustarse a dominios complejos.
- **Ciclo de vida recomendado:** Generalmente requieren un modelo **iterativo-incremental**. El modelo secuencial (cascada) suele considerarse inviable para este enfoque a largo plazo.
- **Conocimiento:** El aprendizaje no es necesariamente exportable a otros proyectos; se debe hacer explícito para convertir habilidades individuales en buenas prácticas grupales.



Criterios para la Selección de un Modelo

No existe un "proceso ideal"; la elección depende de evaluar el **riesgo**, plazo para ejecutar el proyecto, los **requerimientos**, el **equipo** y sus conocimientos técnicos, y la posibilidad de **validar** el trabajo con el **cliente**:

- **Modelo Secuencial (Cascada):**
 - *Cuando usarlo:* Requerimientos claros, exhaustivos y estables; baja incertidumbre; el cliente no tiene prisa por ver el producto.
- **Modelo Iterativo-Incremental (Agilismo):**
 - *Cuando usarlo:* El cliente necesita resultados rápidos (funcionalidades principales primero); requerimientos cambiantes o poco claros al inicio.
- **Modelo Recursivo/Espiral:**
 - *Cuando usarlo:* El objetivo principal es **disminuir riesgos** en sistemas grandes, costosos y extremadamente complejos

Ciclo de Vida	Procesos de desarrollo que lo admite	Características que lo hacen elegible
Secuencial	Definidos	• Alta certeza (baja incertidumbre). • La organización tiene una forma tradicional de trabajar. • No se pueden realizar entregas parciales del software (se debe entregar todo junto). • Eliminar riesgos de planificación.
Iterativo/incremental	Definidos o Empíricos	• Existe incertidumbre. • Volatilidad de requerimientos. • Posibilidad de entregas parciales. • Uso en etapas tempranas del producto.
Recursivo	Definidos	• Mucho control de riesgos en cada iteración. • No se pueden realizar entregas parciales.

Diferentes modelos de ciclo de vida

Code and Fix

Se desarrolla sin especificaciones o diseño y se lo modifica hasta que el cliente este satisfecho. Los cambios se realizan durante el mantenimiento, lo que es muy caro.

Este modelo tiene dos ventajas:

- No se dedica tiempo a planificar, documentar, o cualquiera actividad que no sea codificación pura. Es decir, se salta directamente desde un principio a la codificación y los progresos se ven inmediatamente.
- Requiere poca experiencia, cualquiera que sepa un lenguaje de programación puede familiarizarse con este modelo.

Modelo en cascada puro

Se sigue una secuencia de pasos ordenada y se realiza revisión al final de cada fase. Es conducida por la documentación con fases que no se superponen.

- Permite encontrar errores en etapas tempranas.
- Hay exceso de documentación y falta de resultados hasta el final.
- Se utilizan cuando la definición del producto es estable o los req de calidad dominan a los costos y tiempos.
- Funciona bien cuando se tiene un equipo técnicamente débil o con poca experiencia porque provee al proyecto una estructura que ayuda a minimizar el esfuerzo desperdiciado.
- Este modelo no es flexible, se requiere especificar completamente los requerimientos al comienzo y puede pasar mucho tiempo hasta que se tenga software funcionando. Olvidarse de algo puede ser un error muy costoso.

Modelo en cascada con fases solapadas

Hay un fuerte grado de solapamiento. La documentación es menor, pero es más difícil hacer el seguimiento de progreso.

Modelo de entrega por etapas

Se ven signos tangibles de progreso. Es útil cuando el cliente necesita funcionalidad de inmediato y las necesidades se modifican. Debe haber una planificación rigurosa para que funcione.

Modelo en cascada con retroalimentación

Es una variación del modelo clásico en que es posible volver a una etapa anterior antes de llegar al final, aunque es muy caro hacerlo.

Modelo en cascada con subproyectos

Es un modelo secuencial. Se avanza en cascada hasta el diseño arquitectónico, de ahí en más se avanza en partes dividiendo en Subproyectos.

Modelo en espiral

Busca **minimizar los riesgos** haciendo un análisis de riesgos y buscando alternativas al iniciar cada fase. Al final de cada fase se planifica la siguiente y se evalúa si se sigue adelante. Este modelo parte el proyecto en mini proyectos.

- Permite evaluar la factibilidad de un nuevo producto.
- En proyectos grandes la identificación de los riesgos puede costar más que el desarrollo.
- Es un modelo adecuado para proyectos de vida largos, que puede utilizar equipos diferentes en cada ciclo y muestra el progreso al final de este.
- Por lo general a medida que el costo incrementa, el riesgo disminuye. Mientras mas tiempo y dinero gastas, menos riesgos tomas, que es lo que se quiere en un proyecto de desarrollo rápido.
- Puede ser difícil definir objetivos e hitos que identifiquen que se está listo para avanzar a la siguiente capa del espiral.

Modelo evolutivo

Se fija el tiempo o el alcance mientras el otro se va modificando. Es útil cuando los requerimientos no son claros y se realizan entregas tempranas al cliente.

- El problema es que es un modelo recursivo que repite todas las tareas y depende de que los diseñadores desarrollen un sistema fácil de modificar.

Diseño para cronograma

No se asegura alcanzar la versión final, aunque si una entrega del producto para la fecha definida con los aspectos más importantes. Es útil cuando no hay seguridad sobre las capacidades de programación. 16

Diseño para herramientas

Consiste en incluir solo la funcionalidad soportada por las herramientas de software existentes. Se usa en proyectos muy sensibles al tiempo. Puede combinarse con otros ciclos de vida, pero no se podrán implementar todos los requerimientos.

Prototipación evolutiva

Se da cuando hay cambios rápidos de requerimientos y los clientes no se comprometen con los requisitos. Requiere menos documentación, pero no se pueden programar los release. Se desarrolla el concepto del sistema a medida que se avanza. Es útil cuando los desarrolladores no están seguros de que algoritmos o arquitectura optima utilizar.

El sw evoluciona cambia pueden cambiar cualquiera de sus componentes (código, especificaciones, etc.)

El cambio implica algunas dificultades a la hora de mantener la integridad de ese producto de sw

XQ cambia el sw?

- Cambian los requerimientos de negocio
- Aparezcan defectos
- Soporte de cambios de producto

5 razones para no comparar software y manufactura

- El software es menos predecible
- No hay producción en masa, casi ningún producto de software es igual a otro.
- No todas las fallas son errores
- El software no se gasta
- El software no está gobernado por las leyes de la física

Ciclo de vida	¿En qué consiste?	Ventajas	Desventajas	¿Cuándo conviene elegirlo?
Code and Fix	Se empieza a programar directamente, sin mucha planificación, especificación ni diseño. Luego se corrige hasta que el cliente queda conforme.	Permite ver avances rápidos. No requiere mucha planificación ni documentación. Es simple de aplicar.	Genera mucho retrabajo. Los cambios suelen hacerse en mantenimiento, lo que resulta caro. Puede afectar la calidad y la estructura del software.	Solo en proyectos muy pequeños, simples, de baja criticidad o prototipos informales. No conviene para proyectos grandes o importantes.
Modelo en cascada puro	El proyecto avanza de forma secuencial: requerimientos, análisis, diseño, implementación, pruebas y mantenimiento. Una fase comienza cuando termina la anterior.	Da mucho orden y estructura. Facilita la planificación inicial. Genera documentación clara. Puede ayudar a equipos con poca experiencia.	Es poco flexible. Exige requerimientos claros desde el inicio. El cliente ve software funcionando recién al final. Si aparece un error tarde, corregirlo puede ser muy costoso.	Cuando los requerimientos son claros, estables y completos; hay baja incertidumbre; el producto está bien definido y no se esperan muchos cambios.
Cascada con retroalimentación	Similar a cascada, pero permite volver a una fase anterior si se detecta un problema.	Es más flexible que la cascada pura. Permite corregir errores antes de llegar al final.	Volver atrás puede ser caro y lento. Sigue dependiendo bastante de una planificación inicial fuerte.	Cuando se quiere mantener estructura secuencial, pero aceptando que puede ser necesario revisar fases anteriores.
Cascada con fases solapadas	Las fases se superponen parcialmente. Una etapa puede comenzar antes de que la anterior termine por completo.	Reduce tiempos. Hay menos espera entre equipos. Puede acelerar el desarrollo.	Es más difícil controlar el avance. Puede generar confusión si no hay buena coordinación.	Cuando se necesita acortar plazos, pero los requerimientos siguen siendo relativamente estables.
Cascada con subproyectos	Se avanza en cascada hasta el diseño arquitectónico y luego se divide el desarrollo en subproyectos.	Permite dividir el trabajo. Facilita trabajar por partes una vez definida la arquitectura. Puede mejorar la organización en sistemas grandes.	Requiere una arquitectura clara desde temprano. Puede haber dependencias entre subproyectos.	Cuando el sistema puede dividirse en módulos o partes relativamente independientes después de definir la arquitectura.
Modelo iterativo-incremental	El sistema se construye mediante iteraciones sucesivas. Cada iteración entrega un incremento funcional del producto.	Entrega software útil antes. Permite recibir feedback frecuente. Reduce el costo de adaptar cambios. El cliente participa más. Sirve para requerimientos cambiantes.	Puede invisibilizar el proceso. Documentar cada incremento puede ser costoso. Si no se cuida el diseño, la estructura del software puede degradarse.	Cuando los requerimientos no están totalmente claros, pueden cambiar, o el cliente necesita ver resultados rápidos. Es típico de enfoques ágiles.

Ciclo de vida	¿En qué consiste?	Ventajas	Desventajas	¿Cuándo conviene elegirlo?
Modelo de entrega por etapas	El producto se entrega en partes o versiones sucesivas, con funcionalidades completas en cada etapa.	Permite entregar valor antes del final. El cliente ve progreso tangible. Sirve cuando se necesita funcionalidad rápidamente.	Requiere buena planificación para decidir qué entra en cada entrega. Puede haber problemas si las etapas no están bien priorizadas.	Cuando el cliente necesita funcionalidades pronto y el producto puede dividirse en entregas progresivas.
Modelo evolutivo	Se fija el tiempo o el alcance, mientras el otro aspecto se va ajustando. El sistema evoluciona con entregas tempranas y feedback.	Útil cuando los requerimientos no son claros. Permite adaptación. Favorece entregas tempranas.	Depende de que el sistema sea fácil de modificar. Puede generar desorden si no hay buena arquitectura.	Cuando hay incertidumbre sobre los requerimientos y se necesita aprender mientras se construye.
Prototipación evolutiva	Se construyen prototipos que evolucionan hasta convertirse en el sistema final o en una base para definirlo mejor.	Ayuda a entender requerimientos poco claros. Permite validar ideas con el cliente. Requiere menos documentación inicial.	Puede ser difícil planificar releases. El cliente puede no comprometerse con requisitos definitivos. Puede crecer sin buena estructura.	Cuando los requerimientos cambian rápido, el cliente no sabe exactamente qué quiere, o hay dudas técnicas sobre arquitectura, algoritmos o interfaz.
Modelo en espiral / recursivo	Divide el proyecto en ciclos o “mini proyectos”. En cada vuelta se determinan objetivos, se analizan riesgos, se desarrolla y prueba, y se planifica la siguiente iteración.	Es muy útil para gestionar riesgos. Sirve en sistemas grandes, complejos y costosos. Permite evaluar factibilidad antes de avanzar.	Puede ser costoso. Requiere experiencia para identificar riesgos. Puede ser difícil definir hitos claros para avanzar. En algunos casos se obtiene una versión del producto recién al final.	Cuando el proyecto es grande, complejo, de largo plazo, con alta incertidumbre técnica o muchos riesgos.
Diseño para cronograma	Se fija una fecha de entrega y se priorizan las funcionalidades más importantes para llegar a tiempo.	Asegura una entrega en una fecha definida. Obliga a priorizar lo esencial.	No garantiza alcanzar la versión final completa. Puede quedar funcionalidad afuera.	Cuando la fecha es la restricción principal y se acepta reducir alcance.
Diseño para herramientas	Solo se incluye funcionalidad que pueda desarrollarse con herramientas existentes.	Acelera el desarrollo. Reduce esfuerzo técnico. Sirve en proyectos muy sensibles al tiempo.	No permite implementar todos los requerimientos. El producto queda limitado por las herramientas disponibles.	Cuando el tiempo es crítico y se prioriza entregar rápido sobre cubrir toda la funcionalidad deseada.

Componentes de un proyecto de Sistemas de Información

Proyecto

- Un proyecto de software es un esfuerzo temporal que requiere del acuerdo de un conjunto de especialidades y recursos para la creación de un producto, servicio o resultado único. Es una unidad organizativa para adaptar el marco teórico del proceso, dependiendo de las personas y recursos con los que se cuente.
- Define el proceso y tareas que se van a realizar, el personal que se encargará de las actividades y los mecanismos que se implementarán para valorar riesgos, controlar el cambio y evaluar la calidad.

Características de un proyecto

- Únicos: El resultado de un proyecto será único e irrepetible para cualquier otro proyecto, por más parecido o igual que sean sus elementos componentes.
- No ambiguos: deben ser lo suficientemente claros para guiar el correcto desarrollo del mismo.
- Medibles: se debe poder medir el objetivo para determinar el avance.
- Alcanzable: deben ser factibles de realizarse por el equipo.
- Duración de tiempo limitada: Un proyecto tiene principio y fin, liberando así recursos y personas. Cuando se alcanzan los objetivos del proyecto, el mismo llega al fin.
- Conjunto de tareas interrelacionadas basadas en esfuerzo y recursos: Se definen tareas, dependencias, recursos y personas asignadas.

Administración de Proyectos de Software

- La administración de proyectos es la organización y coordinación de personas y recursos que permiten desarrollar un producto o servicio de acuerdo con el presupuesto, tiempo y funcionalidades acordadas con el cliente, asegurando que se entregue un producto o servicio de alta calidad.
- Es una parte fundamental para que el proyecto sea exitoso.
- Tiene 2 grandes actividades
 - Planificación
 - Monitoreo y control
- La administración incluye:
 1. Identificar los requerimientos
 2. Establecer objetivos claros y alcanzables
 3. Adaptar las especificaciones, planes y el enfoque a los diferentes intereses de los involucrados (stakeholders)
- Una buena gestión no garantiza el éxito del proyecto, pero una mala gestión usualmente sí implica la falla de un proyecto (entrega tardía del SW, costo mayor al estimado, no cumplir con expectativas, etc.)
- Un proyecto puede fracasar, pero lo importante de planificar es el acto en sí y no el resultado.

Las metas más importantes de la administración de proyectos son:

1. Entrega del SW al cliente en el tiempo acordado
2. Mantener los costos dentro del presupuesto general
3. Entregar el SW cumpliendo las expectativas del cliente
4. Mantener un equipo de desarrollo óptimo y con buen funcionamiento

Atributos particulares en la administración de proyectos de SW

1. El producto es intangible: no podemos ver ni tocar el producto que se obtiene al ejecutar el proyecto, dificultando así la medición del progreso.
2. Los proyectos de SW suelen ser excepcionales: los conocimientos adquiridos en un proyecto difícilmente son extrapolables a otro.
3. Los procesos de desarrollo son variables y específicos a la organización: a pesar de querer estandarizar los procesos de desarrollo, no es posible predecir de manera confiable que proceso es el adecuado para adoptar en el proyecto

La triple restricción – enfoque tradicional

- Tenemos 3 variables de restricción en un proyecto tiempo, costo y alcance. El balance entre estos 3 factores afecta directamente a la calidad del proyecto, ya que un proyecto de calidad es aquel que entrega el producto, servicio o resultado requerido, satisfaciendo los objetivos en el tiempo estipulado y con el presupuesto planificado.
- El encargado del balance entre estas 3 restricciones es el LIDER DEL PROYECTO
 - Alcance: son los requerimientos que se incluyen en el proyecto y definen la funcionalidad que va a tener el producto a entregar al cliente. En las metodologías tradicionales es lo primero que se determina y queda fijo en función de las necesidades del cliente.
 - Tiempo: Define la fecha calendario en al cual se compromete a finalizar el proyecto y entregar el producto resultante al cliente. En metodologías ágiles el tiempo es FIJO.
 - Costo: define el costo de todos los recursos implicados en el desarrollo del proyecto (equipamiento, infraestructura, equipos, salarios, etc.)
- En la realidad, el costo y el tiempo del proyecto varían de forma directa con la definición del alcance del producto, es decir, el alcance es directamente proporcional al tiempo y al costo.

Líder de Proyecto

- Es quien realiza las tomas de decisiones en la totalidad, sin incluir opiniones del equipo. Es quien asigna las tareas que deben realizar los integrantes del equipo.
- Maneja todas las relaciones con los stakeholders del proyecto (clientes, gerencias, contratistas, etc.)
- Los profesionales técnicos no disponen de habilidades para tratar con la gente, por lo que es necesario una figura de líder de proyecto, que cuente con ciertas aptitudes.
 - Motivación
 - Organización
 - Innovación
 - Empatía
 - Comunicación
 - Liderazgo
- También cuenta con habilidades duras, relacionadas a los conocimientos del producto, técnicas y herramientas. Ya que tiene un enfoque basado en la resolución de problemas.
 - Resolución de problemas
 - Identidad administrativa
 - Logro
 - Influencia
- Tiene como responsabilidades
 - Definir alcance del proyecto
 - Identificar a los involucrados, recursos y presupuesto
 - Detallar tareas, estimar los tiempos y requerimientos
 - Identificar y evaluar riesgos
 - Preparar planes de contingencia
 - Controlar hitos y participar en las revisiones de las fases del proyecto
 - Administrar el proceso de control de cambios
 - Producir reportes de estado

Equipo de proyecto

- Grupo de personas comprometidas con alcanzar un conjunto de objetivos. Algunas de sus características son:
 - Cuentan conocimientos, habilidades técnicas, motivación, experiencias y diversas personalidades. Tienen sentimiento de mejora continua y aprendizaje mutuo.
 - Usualmente son un grupo pequeño, logrando mejor comunicación afectiva entre ellos.
 - Trabajo en conjunto, con espíritu de grupo. Convirtiéndose en un equipo cohesivo.
 - Sentido de responsabilidad como una unidad y focalización en el cumplimiento de metas grupales.
- En equipos cohesivos:

- El grupo establece sus propios estándares de calidad
- Los individuos aprenden de los demás y se apoyan mutuamente
- El conocimiento se comparte
- Prevalece la mejora continua

Stakeholders

- Son los interesados del proyecto, incluye el equipo del proyecto, equipo de dirección, líder del proyecto y el patrocinador.

Plan de proyecto

- Es un artefacto que cumple la función de hoja de ruta de un proyecto, en la cual se documentan todas las decisiones que se toman a lo largo del desarrollo, en términos del proceso, división de tareas, asignación de responsabilidades, equipos de trabajo, calendario de desarrollo actividades de soporte.
- En un plan de proyecto se establece que se va a hacer (alcance), cuando se va a hacer (calendario), como se va a hacer (actividades o tareas para cumplir el alcance) y quien lo va a hacer (responsables de cubrir las actividades).
- Para realizarlo se deben seguir estas actividades:
 - Definición del alcance del proyecto
 - Definición de proceso y ciclo de vida
 - Estimación
 - Gestión de riesgos
 - Asignación de recursos
 - Programación de proyectos
 - Definición de métricas
 - Definición de controles

1. Definición del alcance del proyecto

- Responde al que se está desarrollando. Es importante diferenciar el alcance de un producto y del del proyecto en sí.
- El alcance de un producto define que funcionalidades debe realizar el SW para satisfacer los requerimientos del cliente y se mide contra la ERS.
- El alcance del proyecto define todo el trabajo necesario para cumplir con el desarrollo de este y poder entregar el producto o servicio con todas las características y funcionalidades especificadas. Se mide contra el plan de proyecto.
- Tanto el alcance como el objetivo de un proyecto puede ser modificado a lo largo del proyecto, pero esto no necesariamente tiene que cambiar el alcance del producto.
- El alcance del proyecto es menor que el alcance del producto.

2. Definición del proceso y ciclo de vida

- Al definir el proceso que se va a utilizar, se responde al cómo se va a desarrollar el proyecto. Es importante tener en cuenta la información del contexto del desarrollo: objetivos, alcances, personas y recursos disponibles, clientes, forma de trabajo de personas, etc.
- Se debe tener en cuenta la complejidad del software a desarrollar (alcances y objetivos) para así poder adaptar el proceso definido que se elija al proyecto en el cual se está trabajando.
- El ciclo de vida del proyecto es lo que define cuánto de cada tarea se debe hacer y en qué momento del desarrollo del proyecto hacerlo. El ciclo de vida define lo siguiente:
 - Que trabajo debería realizarse en cada fase
 - Quien debería estar involucrado en cada fase
 - Como controlar y aprobar cada fase
 - Como deben generarse los entregables
 - Como revisar, verificar y validar el producto.

- En metodologías tradicionales se puede elegir cualquier tipo de ciclo de vida (cascada, iterativo e incremental, espiral, etc.). En metodologías ágiles si o si debe ser iterativo e incremental

3. Estimación

- En un proyecto de desarrollo de software, se trata de estimar para predecir el valor de un elemento relacionado con el proyecto o con el producto, por ejemplo, el tiempo que va a llevar, qué costo va a tener el sistema, cuántas personas necesito para realizar el desarrollo, etc.
- En la metodología tradicional, existe un orden definido y recomendado para realizar las estimaciones:
 1. Tamaño de producto
 2. Esfuerzo
 3. Calendario
 4. Costos

4. Gestión de riesgos

- Un **riesgo** es la probabilidad de ocurrencia de una pérdida o daño que afecte de forma directa al proyecto y pueda comprometer el incumplimiento de su objetivo.
- Los podemos clasificar en:
 1. Riesgos del proyecto: alteran el calendario o recursos del proyecto.
 2. Riesgos del producto: afectan la calidad o el rendimiento del SW a desarrollar.
 3. Riesgos organizacionales o de negocio: afectan a la organización que desarrolla o que adquiere el SW.
- Para medir los riesgos se utiliza lo que se conoce como exposición al riesgo (probabilidad x impacto), la cual se obtiene como el **producto entre la probabilidad de ocurrencia de la amenaza por el impacto que generaría si realmente ocurriera**. Este valor permite organizar y gestionar los riesgos, y se debe hacer foco en reducir la exposición del riesgo.
- Ante los riesgos es posible tomar 3 posturas/actitudes:
 - Negación: hacer de cuenta que los riesgos no existen.
 - Gestión reactiva: esperar a que el riesgo ocurra y recién ahí ver que hacer.
 - Gestión proactiva: ir trabajando sobre el riesgo, generando planes de mitigación y de contingencia.
- Entre los mayores riesgos en el entorno de desarrollo de software se encuentran:
 - Cambio de requerimientos en el medio de un desarrollo
 - Abandono de integrantes del equipo de desarrollo
 - Falta de capacidad del equipo
 - El gran avance tecnológico
- Actividades para gestionar los riesgos:
 1. Identificación de riesgos: se identifican los riesgos que suponen una mayor amenaza al proceso de ingeniería de software, al software a desarrollar o a la organización que lo desarrolla.
 2. Análisis de riesgos: para cada riesgo identificado se realiza un juicio acerca de la probabilidad y del impacto. El criterio dependerá de la experiencia de quien realice el análisis de riesgos.
 3. Planeación del riesgo: para cada riesgo analizado, se definen estrategias para manejarlos. Consisten en considerar acciones a tomar para minimizar o evitar el impacto sobre el proyecto como consecuencia de la ocurrencia de la amenaza que representa el riesgo.
 4. Monitorización del riesgo y control: proceso que permite determinar que las suposiciones acerca de los riesgos de producto, proceso y organización no han cambiado. Permite revalorar al riesgo en términos de posibles variaciones de su probabilidad e impacto.

5. Asignación de responsabilidades (o recursos)

- La asignación de responsabilidades a personas permite asignar roles a los integrantes del equipo de trabajo.
- Dentro de un mismo proyecto, una persona puede desenvolverse en más de un rol, ya que puede que el presupuesto del desarrollo de software no sea lo suficientemente grande para permitirse tener una persona distinta por rol, o bien, el proyecto no sea lo suficientemente grande o complejo para justificar dicha adición.
- Para dejar explícitos los roles y quienes son las personas que los cumplen, generalmente se construye una tabla, la cual contiene la información de la persona, el rol y responsabilidades de esta en el contexto del proyecto.

6. Programación de proyectos (calendarización)

- Se trata de definir en detalle, cada tarea que debe ser cumplida en el desarrollo. Se toma como base lo definido en la estimación del calendario, pero se refina al máximo detalle posible
- Cuando se habla de detallar las tareas, se hace referencia a descomponer el proceso de desarrollo en actividades refinadas a nivel de días, identificando quien la realiza, cuando debe realizarse y cuánto esfuerzo debe llevar en forma teórica. La calendarización se realiza a través de un Diagrama de Gantt

7. Definición de métricas

- Se definen como una medida numérica que aporta visibilidad sobre el avance o estado del proyecto, proceso o producto en un momento determinado del desarrollo.
- Al realizar la planificación del proyecto, es necesario definir de forma clara y no ambigua, cada una de las métricas asociadas a cada dominio (producto, proyecto o proceso) en conjunto con la forma de calcularlas.
- Deben representar un aumento en la información y benefician la practicidad del desarrollo y seguimiento.
- A diferencia de las metodologías ágiles, acá se definen métricas que miden y exponen los avances. En cambio, en ágil, el mayor indicador de avance de un producto, son los incrementos que se entregan en cada iteración.
- Las métricas básicas para un proyecto de software se clasifican en:
 - Tamaño del producto
 - Esfuerzo
 - Calendario (tiempo)
 - Defectos
 - Costos

8. Definición de controles

- En la planificación de los controles del desarrollo, se toma como base las métricas ya definidas, y verifica que todo se está haciendo en concordancia con lo establecido.
- En esta planificación se definen reuniones periódicas, reportes o informes necesarios, etc.

Planes de soporte

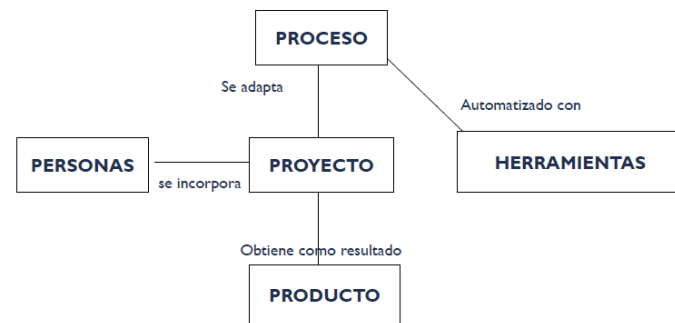
- Planes de Testing.
- Planes de Mantenimiento.
- Planes de subcontrataciones.
- Planes de calidad.
- Planes de capacitaciones.
- Planes de iteraciones (si el ciclo de vida es iterativo/incremental).

Causas de fracasos de proyectos

- Fallas de toma de REQUERIMIENTOS (el 80%).
- Fallas al definir el problema.
- Planificar basado en datos insuficientes.
- La planificación la hizo el grupo de planificaciones.
- No hay seguimiento del plan de proyecto.
- Plan de proyecto pobre en detalles.
- Planificación de recursos inadecuada.
- Las estimaciones se basaron en “supuestos” sin consultar datos históricos.
- Nadie estaba a cargo.

Vinculo proceso-proyecto-producto en la gestión de un proyecto de desarrollo de software

- Se dice que el proceso, automatizado con herramientas, se adapta al proyecto, al cual se incorporan personas, y de este se obtiene como resultado un producto.
- En el enfoque tradicional, previo a la definición del proyecto, es necesario determinar los objetivos y el ámbito del producto para de esta manera poder realizar las estimaciones pertinentes.
- Los objetivos identifican las metas globales del producto sin especificar como se van a lograr. El ámbito del producto define las funciones y comportamientos que caracterizan al producto que utilizara cliente.
- Para obtener como resultado un producto o servicio, es necesario que el proyecto adopte un proceso de desarrollo para poder definir qué actividades, métodos, prácticas y transformaciones se utilizarán, mediante las cuales se va a obtener el software. Dentro de cada proyecto de desarrollo de SW es necesario utilizar un proceso diferente dependiendo del tipo de producto que se va a desarrollar.
- El proceso es una definición teórica de lo que debería hacerse para tomar los requerimientos y transformarlos en un producto o servicio, guiados por el objetivo de lo que se quiera construir.
- Las personas son la parte más importante, debido a que el desarrollo de software es una actividad humano-intensiva. Si se adopta el “mejor proceso” para el desarrollo de un software, y además se destina mucho esfuerzo a planificar el proyecto de desarrollo, de nada servirá si no tenemos un equipo capacitado que reúna las habilidades y herramientas necesarias para el desarrollo del mismo.



El producto es lo que se quiere construir, por eso primero se definen sus objetivos y su ámbito: qué metas tiene, qué funciones tendrá y qué comportamiento espera el cliente.

El proyecto es el esfuerzo organizado para desarrollar ese producto. Para llevarlo adelante, necesita adoptar un proceso de desarrollo, es decir, una forma de trabajo con actividades, métodos, prácticas y herramientas.

El proceso guía cómo transformar los requerimientos en software, pero debe adaptarse al tipo de proyecto y producto. No existe un único proceso ideal para todos los casos.

Proceso + herramientas + personas → se aplican al proyecto → para obtener un producto.

Lo más importante es que las personas son el factor central, porque el desarrollo de software es una actividad humano-intensiva. Aunque se use un buen proceso y se planifique bien, el proyecto no tendrá éxito si el equipo no está capacitado, motivado y no cuenta con las herramientas necesarias.

El producto define qué se quiere lograr, el proyecto organiza cómo hacerlo y el proceso indica qué actividades seguir para transformar requerimientos en software.

NO SILVER BULLET - PAPER

No Silver Bullet" de Frederick P. Brooks, Jr.: Dificultades Esenciales y Accidentales en Ingeniería de Software y Posibles Soluciones

Contrastar progreso del hardware (algo tangible, construible, que se puede mejorar en su construcción), en comparación con el software (intangible, complejo inherentemente)

Las balas de plata son aquellas soluciones que podrían hacer frente a la crisis del software: no existe un desarrollo de software que demuestre avances de progresión (simplicidad, fiabilidad y productividad) en la construcción del software en menos de una década.

Dificultades en el Desarrollo de Software

Dificultades Esenciales: todo aquello que trae aparejado el construir software: conceptos abstractos, estructuras de datos, algoritmos, invocación de funciones, etc. (carece de precisión y detalles). 4 aspectos la caracterizan: ■
Complejidad:

- Propiedad inherente, debido a la abstracción del software
- Al crecer el tamaño del software, su complejidad crece de manera exponencial, no lineal
- Esto conduce a muchos problemas: dificultad de comunicación entre miembros de equipos, deficiencias en el producto, errores en cumplimiento de fechas, excesos en costos, etc.

Conformidad: El software debe realizar aquello que el cliente necesita, esto es: debe adecuarse a sus requerimientos.

Mutabilidad: El software está sujeto a constantes cambios. A diferencia de objetos tangibles, el software muta y sufre cambios a lo largo del tiempo. En cambio, en muchos objetos esto no es así, ya que se reemplazan por nuevos productos (no tomo el objeto y lo modifico, creo uno nuevo).

Invisibilidad: Al ser algo no visualizable, se dificulta mucho trabajar con representaciones geométricas, como puede ser con estructuras mecánicas. Los grafos son una buena herramienta que permiten modelar muchas situaciones de un software.

Dificultades Accidentales: tiene que ver con aquellos aspectos que dificultan la construcción del software, pero han sido solucionados a lo largo del tiempo. Esas soluciones son:

- Lenguajes de programación de alto nivel: permiten abstraer el proceso de construcción del software, independizándose del hardware en el que se ejecutarán. Esto elimina la complejidad propia de una máquina, que nada tiene que ver con el software que se desea construir.
- Tiempo compartido: reducción de los tiempos de respuesta de un software. Esto permite no desviar el foco de lo que se desea construir.
- Entornos de programación unificados: facilitan la construcción del software al integrar librerías, formato de archivos, frameworks, etc.

Resolución de dificultades accidentales

- Programación orientada a objetos: permite a los diseñadores del software, abstraerse de tecnicismos para expresar su diseño simplemente en un lenguaje de máquina. Esto facilita el diseño, pero sólo elimina una dificultad accidental, no elimina la propia dificultad esencial de diseñar software.
- Inteligencia Artificial: si bien la IA permite resolver muchas cuestiones que reemplazan acciones humanas, siempre tiene que haber un equipo de personas que programe y “enseñe” a esa IA cómo actuar. Con lo cual, el factor humano no se reemplaza del todo.
- Sistemas expertos: son sistemas que aplican la IA y permiten sugerir diferentes situaciones como estrategias de Testing, optimizaciones de código, sugerencias en nombrado de variables, etc. Igualmente, no se quita el factor humano ya que se necesita encontrar expertos y desarrollar técnicas eficientes para extraer lo que ellos saben para destilarlo dentro de bases de reglas de estos sistemas. El prerequisite esencial para construir un sistema experto es tener un experto
- Programación automática: esto se planteó como un reemplazo a los programadores, pero en lugar de ser eso, terminan siendo un lenguaje de más alto nivel que el programador debe parametrizar.
- Programación gráfica
 - Permite programar a un nivel más alto de abstracción que la programación tradicional, pero sigue estando el factor humano presente, con lo que solo se logra dar una mayor abstracción al programador sin que este tenga la necesidad de aprender sintaxis del lenguaje.
- Verificación de programas:
 - Lo que se logra con la verificación de programa, es establecer que un programa pasa las pruebas matemáticas de verificación que se proveen.
 - No permite verificar que se cumplan todas las necesidades del cliente que fueron previstas en el software en forma de funcionalidades, sino que verifica que las funcionalidades desarrolladas no tengan errores.
 - Además, las pruebas matemáticas pueden también estar sometidas a errores.
- Environments and tools
 - La mayoría de los nuevos entornos inteligentes tan solo nos permiten librarnos de errores semánticos y sintácticos simples.
- Estaciones de trabajo poderosas
 - El incremento de potencia y memoria de las estaciones de trabajo individuales no soluciona el déficit en el desarrollo el software.
 - La edición de programas y documentos son soportadas totalmente por las máquinas de hoy en día.
 - Una mejora en tiempos de compilación no representa un avance significativo.

Resolución de dificultades esenciales

- Comprar, antes que construir: explorar en el mercado soluciones que se puedan conseguir para reemplazar lo que se quiera construir. Si existe un mercado que lo ha demandado, comprarlo siempre será más barato que construirlo. Esto se debe a que al tener una gran cantidad de clientes que lo compran, el costo de ese software se amortiza. Mirarlo como que el uso de N copias de software, multiplica la productividad de ese software N veces.
- Lógicamente este enfoque es difícil de utilizar para necesidades muy puntuales, donde no existe una porción de mercado que lo haya demandado previamente.
- Usar prototipos y refinar requerimientos: la parte fundamental de construir software es saber qué se debe construir. La extracción iterativa de requerimientos y su posterior refinamiento permiten tomar las necesidades de los clientes en sucesivas iteraciones, debido a que ellos mismos no saben expresar ni conocer muchas veces cuáles son esas necesidades que tienen. Además, es prácticamente imposible que sepan expresar con detalle y precisión todos sus requerimientos para el software que necesitan. El desarrollo de prototipos rápidos permite cubrir esa extracción iterativa de requerimientos, al simular interfaces y funciones principales de un software que se necesita construir. Esto permite obtener un feedback valioso del cliente, para que todos comprendan mejor cuáles son las funcionalidades que se necesitan, y cómo se deben ejecutar.
- Desarrollo incremental: (crecer y no construir software): Incrementar el software añadiendo funcionalidades mientras estas se prueban, usan y testean.

Tener un software con pocas funcionalidades, pero funcionando, es más animador que tener un avance de un software complejo que no funciona.

- Personas capacitadas: que sigan buenas prácticas. Esto permite tener un buen equipo, pero, al fin y al cabo, desarrollar software requiere de creatividad, por lo que priorizar la creatividad en un equipo puede traducirse en grandes resultados (estructuras rápidas, pequeñas, simples construidas con menos esfuerzo).

Esto plantea que tener en un equipo con buenos diseñadores, es tan importante como tener buenos líderes que dirijan ese equipo. Por lo que se debe recompensarlos de la misma forma, debido a su gran valor para el desarrollo del software.

Otras Técnicas con Impactos Limitados

Brooks analiza brevemente varias áreas consideradas "balas de plata" potenciales:

- Lenguajes avanzados y programación orientada a objetos: Mejoran la modularidad y expresión del diseño, pero sólo atacan dificultades accidentales, por lo que las mejoras no serán dramáticas.
- Inteligencia artificial y sistemas expertos: Pueden asistir en tareas específicas, como la generación de pruebas o debug, pero no resolverán el problema esencial del diseño conceptual.
- Programación gráfica y herramientas visuales: La invisibilidad del software dificulta enfoques gráficos amplios y efectivos; además, el espacio visual limitado y la complejidad no lineal hacen inviable la visualización global.
- Verificación formal: Importante para ciertos sistemas críticos, pero no elimina la dificultad principal que es la correcta especificación y diseño del software.

Unidad 2: Gestión Lean-Agile de Productos de Software

Manifiesto ágil → documento redactado en 2001 por expertos de la Ingeniería en Software, que promueve la filosofía del desarrollo ágil del software.

¿Qué es Ágil? → NO es una metodología o proceso – tenemos de antes un concepto erróneo de metodologías ágiles justamente al ser AGILES no podemos seguir un método, pasos, condiciones para poder avanzar, etc.

¿Pero qué significa Ágil?

- Balance entre ningún proceso y demasiado proceso. La diferencia inmediata es la exigencia de una menor cantidad de documentación, sin embargo, no es eso lo más importante:
 - Los métodos ágiles son adaptables en lugar de predictivos.
 - Los métodos ágiles son orientados a la gente en lugar de orientados al proceso
- El cliente forma parte de nuestro equipo de desarrollo

Filosofía Ágil

El objetivo primordial es entregar software de forma frecuente en un entorno cambiante.

Estas metodologías se utilizan en entornos con gran variabilidad de requerimientos, involucrando al cliente en el proceso de desarrollo para conseguir una rápida retroalimentación.

El desarrollo ágil adopta el ciclo de vida iterativo-incremental, donde el concepto del empirismo subyace al proceso, el software no se desarrolla como una sola unidad, sino como una serie de incrementos y cada uno de ellos incluye una nueva funcionalidad del sistema.

Los 3 pilares del empirismo/ Agilismo

1. **Transparencia** → nos permite crecer como equipos y transformar el conocimiento que cada miembro tiene en conocimiento de todo el equipo. “no es mi código, es nuestro código”. Todo lo que desarrollemos y nos pase dentro del equipo, pasa a ser parte de todos.
Los artefactos que tienen poca transparencia pueden conducir a decisiones que disminuyen el valor y aumentan el riesgo.
La transparencia permite la inspección. La inspección sin transparencia genera engaños y desperdicios.
2. **Inspección** → La inspección puede realizarse para el producto, procesos, aspectos de personas, prácticas y mejoras continuas. Se realiza por todo el equipo y permite detectar varianzas o problemas potencialmente indeseables, se debe inspeccionar el progreso hacia los objetivos acordados.
Es el momento de aprender cosas nuevas que van surgiendo del trabajo que el equipo va realizando.
La inspección permite la adaptación. La inspección sin la adaptación se considera inútil
3. **Adaptación** → Se trata de la mejora continua, es la capacidad de adaptarse en función de los resultados de la inspección. Nos permite hacer ajustes en aspectos del proceso que se desvían fuera de los límites o si el producto es inaceptable.
Los ajustes deben realizarse lo antes posibles para minimizar la desviación normal. El equipo debe adaptarse en el momento que aprenda algo nuevo por medio de la inspección.

¿Cuándo aplicar Ágil?

Es más aplicable en **contextos complejos**, donde no hay total certeza sobre los requerimientos (por ejemplo, desarrollo de productos nuevos). En cambio, en contextos más simples o conocidos (como mantenimiento), no siempre es necesario.

Valores ágiles (4)

1. Individuos en interacciones, por sobre procesos y herramientas: Es preferible un equipo motivado y funcional con comunicación fluida, aunque tenga herramientas limitadas, que uno desmotivado con grandes recursos. Esto se debe a que **el desarrollo de software es una actividad humano-intensiva**, donde lo clave es potenciar las fortalezas individuales y lograr buena comunicación. Las herramientas y procesos deben adaptarse al equipo, no al revés.

2. Software funcionando por sobre documentación extensiva: asegura que se tengan versiones estables, incrementales y mejoradas del software al finalizar cada iteración. Esto permite que a medida que el software crezca en cuanto a funcionalidades, cada iteración puede ir mostrándose a los usuarios finales y obtener una retroalimentación, para que el equipo de desarrollo se asegure de estar trabajando en aquellas funcionalidades de mayor valor para el cliente.

Solo se documenta aquello que agregue valor al producto y esté centrado en el cliente (valor agregado). Previamente se define QUE es lo que se va a documentar y COMO es que se hará, el enfoque ágil lo que plantea es que **se debe generar la documentación cuándo sea necesario. No plantea “no generar documentación”**.

3. Colaboración con el cliente por sobre un contrato firmado: Implica comprometer al cliente como parte importante del producto que se está construyendo.

No se trata de tener a un cliente lejano que no sabe lo que sucede durante el desarrollo, si no tener a uno que esté comprometido con cada entrega, sobre todo al momento de probarlo y dar el feedback.

Permite entender mejor los requerimientos del cliente y fomenta el trabajo conjunto del equipo ágil con los interesados, reduciendo la necesidad de documentación extensa. Esto no quiere decir que no planifiquemos, sino que en vez de hacerlo de acá a un año lo haremos de acá a un sprint

4. Responder a cambios por sobre seguir un plan: esto se plantea debido a que es imposible que los usuarios sepan con detalle todas las funcionalidades que necesitan, y a su vez, con el paso del tiempo surgirán nuevas necesidades.

Principios del Manifiesto Ágil

1. Satisfacción del cliente: se busca entregar valor constantemente según lo que el cliente necesita, priorizado por el Product Owner. Ligado al valor 3 y 4
2. Aceptar cambios: el equipo es flexible y adapta el producto incluso en etapas avanzadas, respondiendo a nuevas necesidades. Ligado al valor 3 y 4
3. Entregas de SW funcional frecuentes: se entrega software funcional en ciclos cortos, completamente terminado y listo para usarse. Ligado al valor 2
4. Trabajo conjunto: técnicos y personas de negocio colaboran todo el tiempo, guiados por el Product Owner. Ligado al valor 1 y 3. En un enfoque de proceso ágil, la participación del Product Owner es de suma importancia, ya que es quien guía y prioriza tras cada iteración.
5. Personas motivadas: el equipo debe estar comprometido y trabajar como un todo, no como individuos aislados. 1
6. Comunicación directa: la forma más efectiva de comunicación es cara a cara, por su mayor riqueza de info. 1
7. Progreso = software funcionando: el avance se mide por lo que realmente funciona, no por documentación. 2
8. Desarrollo sostenible: se busca mantener un ritmo de trabajo constante y equipos estables en el tiempo. 2 y 4
9. Calidad continua: el buen diseño y la excelencia técnica son fundamentales y no negociables. 2
10. Simplicidad es Esencial: se hace solo lo necesario para aportar valor, evitando trabajo innecesario. 2 y 3
11. Equipos autoorganizados: el equipo toma decisiones y propone soluciones, generando mejores resultados. 1
12. Mejora continua: el equipo se evalúa continuamente y ajusta su forma de trabajar (inspección y adaptación). 1 y 4

Filosofía Ágil

El objetivo primordial es entregar software de forma frecuente en un entorno cambiante. Estas metodologías se utilizan en entornos con gran variabilidad de requerimientos, involucrando al cliente en el proceso de desarrollo para conseguir una rápida retroalimentación, ya que estos encuentran imposible predecir cómo un sistema afectará sus prácticas operacionales o qué cambios habrá en el entorno (mercado o políticas de negocio) que pueden dejar el sistema completamente obsoleto.

El desarrollo ágil adopta el ciclo de vida iterativo-incremental, donde el concepto del empirismo subyace al proceso, el software no se desarrolla como una sola unidad, sino como una serie de incrementos y cada uno de ellos incluye una nueva funcionalidad del sistema. Esta es una de las diferencias con el PUD o el RUP, que, si bien utilizan un modelo de proceso iterativo - incremental, son procesos definidos.

Usualmente los incrementos se crean cada 2 o 3 semanas y se ponen a disposición del cliente. El cliente se encuentra inmerso en el proceso, ofreciendo una rápida retroalimentación que permiten una inspección y adaptación sobre los requerimientos cambiantes. Por otro lado, se reduce la cantidad de documentación con el uso de comunicaciones frecuentes e informales con el cliente en lugar de reuniones formales con documentos escritos.

Martin Fowler define al enfoque ágil como “un compromiso entre nada de proceso y demasiado proceso”.

Los métodos ágiles son adaptables en lugar de predictivos y están orientados a la gente en lugar del proceso.

Nace buscando un equilibrio entre hacer Software siguiendo rigurosamente un proceso donde a la larga termina siendo más importante cumplir con el proceso que entregar software de calidad y, yéndonos al otro extremo, hacer software a la ligera.

La filosofía ágil se materializa en el manifiesto ágil. Éste está basado en que la forma de trabajo va a ser sustentada en los conceptos de los procesos empíricos, bajo 3 pilares fundamentales, la transparencia, la inspección y la adaptación.

Filosofía Lean

Es una filosofía o enfoque más viejo que agile, que nace en Japón ('40) con Toyota, y busca maximizar el valor generado al cliente con el mínimo consumo de recursos, argumentando que todo el esfuerzo que no cree valor, se considera desperdicio. Esto permite reducir los esfuerzos en crear artefactos que no agreguen valor a un producto, concentrando los esfuerzos en los aspectos esenciales.

La filosofía "Lean", conduce a una visión integrada de la cultura y la estrategia para atender al cliente final con alta calidad, bajo costo y tiempo de entrega, produciendo exactamente lo que el cliente final quiere, cuando lo quiere, dónde lo quiere, a un costo mínimo y precio justo. Siendo el cliente quien determina si el servicio o producto que la empresa entrega tiene valor o no.

Para esto utiliza el método Just in Time, el cual permite el intercambio del producto entre fases del proceso de desarrollo o a la entrega del producto final al cliente justo a tiempo, esto es poco antes de que lo usen y en la cantidad justa y necesaria, "analice cuando lo necesite, no antes".

7 principios Lean

Estos principios apuntan al desarrollo de productos en la industria de manufactura. Como el software es una actividad de desarrollo de un producto intangible, a los principios originales de Lean, hubo que adaptarlos al desarrollo de software

1. Eliminar desperdicios

el + importante. Debemos tener un proceso que sea eficiente y que sume valor en todos los pasos del proceso, todo paso o parte que no contribuye a la generación de valor produce un desperdicio.

Lo que se busca evitar:

- Producir funcionalidades en exceso, que luego no se usan.
- Retrabajo.
- Que los artefactos se vuelvan obsoletos antes de terminarlas.

Este principio se relaciona con los principios agile 3 y 7 (entregar software funcionando frecuentemente y el software funcionando es la principal medida de progreso) y 10 (la simplicidad o maximizar el trabajo no realizado es esencial)

Los 7 desperdicios de Lean → Se pondrán los desperdicios para SW y entre corchetes para la industria manufacturera.

- **Características extras [producir productos extras]**
- Trabajo a medias [acumulación de stock]
- Proceso extra
- Búsqueda de información
- Defectos
- **Esperas [costos de logística]**
- Cambios de tareas (Multitasking)

2. Amplificar el aprendizaje

Tiene que ver con la transparencia y transformación de conocimiento implícito en explícito, lo cual fortalece al equipo de trabajo.

Crear y mantener un mejoramiento continuo, donde los individuos intercambien sus experiencias y conocimientos para contribuir con el aprendizaje colectivo

3. Embeber la integridad conceptual

La calidad del producto no se negocia y la simplicidad es un factor clave (principios Agile): el producto tiene que satisfacer las expectativas y necesidades del cliente, esa es la principal prioridad y cada decisión se alinea con esto

4. Diferir compromisos hasta el último momento responsable

Las decisiones se toman lo suficientemente “tarde” como para poder tener la mayor información necesaria, pero a la vez, antes de que ese momento sea muy tarde (diferir la decisión irreversible).

5. Dar poder al equipo

Otorgar libertad de acción y poder de decisión al equipo. Para ello, el equipo debe ser multifuncional y autogestionado, y sus miembros deben estar capacitados y motivados.

6. Ver el todo

En Lean se busca tener una visión holística, que permita asociar y comprender el todo, el producto, el valor agregado que hay detrás, el servicio que brinda el complemento de los productos, más allá de los objetivos.

7. Entregar lo antes posible

La idea es entregarle al cliente software funcionando lo más rápido posible y que este producto sea útil para él.

Relación Lean-Ágil

Lean es previo al Manifiesto Ágil. Algunos principios de Agile heredan de los fundamentos de Lean. Ambos están orientados al cliente, a proveer el máximo valor posible. Para esto, entienden que los individuos de los equipos deben estar motivados, de forma tal que la sinergia de este facilite el desarrollo del proyecto.

Esta sinergia, se interpreta como un valor agregado para el cliente. La flexibilidad para adaptarse a los cambios y ofrecer valor al cliente, es también un denominador común de ambos enfoques. Otro aspecto que tienen en común es generar productos de calidad y de mejora continua.

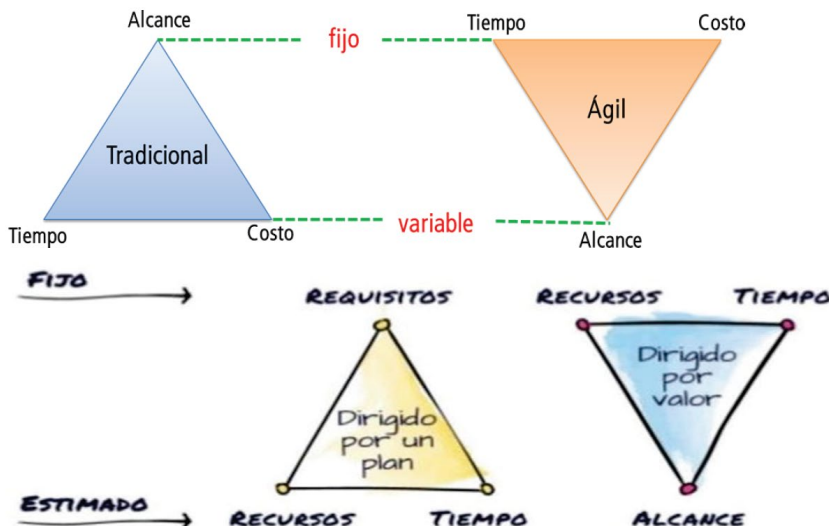
Requerimientos en ambientes ágiles – User Stories

A diferencia de un req en un entorno tradicional los definimos según los CASOS DE USO planteados desde la solución

En el AGILISMO especificamos los req a través de HISTORIAS DE USUARIO ósea que se escriben desde el lado de un usuario, desde la persona que va a hacer USO de esa funcionalidad

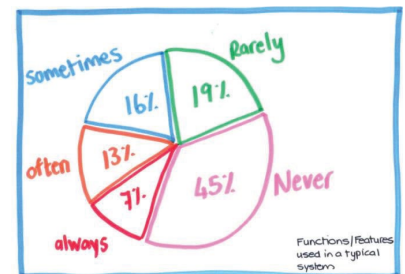
Los requerimientos ágiles se basan en 3 pilares.

- Usar el valor al cliente para construir el producto correcto
- Determinar que es “sólo lo suficiente”: hace referencia a que los requerimientos deben ir descubriéndose
- Usar historias y modelos para mostrar que construir



BRUF – Big Requirements Up Front

La imagen adjuntada expresa la frecuencia con la que los usuarios utilizan determinadas funcionalidades de un software. A lo que se apunta con este concepto de BRUF es primero desarrollar sólo unas pocas funcionalidades de gran valor e importancia para el cliente, puede cubrir gran parte de sus necesidades, a medida que se desarrolla el resto de los requerimientos.



Product Backlog

El Product Backlog es una lista priorizada de características y requerimientos del software.

Es el Product Owner quien define esta lista ya que es él quien tiene en claro cuáles son sus necesidades y requerimientos principales. El Product Owner se va a encargar de decidir qué requerimientos necesita primero según el valor que éstos tengan para el negocio.

En ágil es el cliente el encargado de esta priorización, lo que lleva a que se construya el producto deseado.

Requerimientos Just in Time

La gestión ágil de requerimientos sostiene los **requerimientos justo a tiempo**, asociados a los principios de eliminar el desperdicio y diferir compromisos, dentro de la filosofía Lean.

Se empieza con una visión del producto y con algunos requerimientos identificados. A partir de esta base, se comienza a construir el producto y se incorpora al cliente para obtener retroalimentación. **El Product Backlog nunca está al 100%**

Los requisitos cambiantes son una ventaja competitiva si puede actuar sobre ellos



Tipos de requerimientos

A nivel de la gestión ágil de requerimientos se trabaja con

- Requerimientos de negocio
- Requerimientos de usuario

Las USER STORIES son requerimientos de usuario alineados a un requerimiento de negocio

¿Qué debemos asumir?

- Los cambios son la única constante. Todo el tiempo va a haber cambios en los requerimientos.
- Stakeholders: no son todos los que están, debemos identificar a los involucrados que son los que tienen algo que decir sobre el producto, El product owner es el representante de los mismos y es el que se comunica con ellas para luego comunicarse con el equipo que hace software.
- Siempre se cumple eso de que: "El usuario dice lo que quiere cuando recibe lo que pidió".
- No hay técnicas ni herramientas que sirvan para todos los casos. Debemos detectar para c/situación la adecuada.
- Lo importante no es entregar una salida, un req, lo importante es entregar, un resultado, una solución de valor.

Principios ágiles relacionados con requerimientos ágiles



User Stories

Son una técnica para trabajar requerimientos de usuario en ambientes ágiles.

La US contiene una descripción corta de una funcionalidad que se espera del producto valorada por un usuario del sistema.

La parte más difícil de construir un sistema de software es decidir precisamente qué construir. Ninguna otra parte del trabajo conceptual es tan difícil como establecer los requerimientos técnicos detallados... Ninguna otra parte del trabajo afecta tanto el sistema resultante si se hace incorrectamente. Ninguna otra parte es tan difícil de rectificar más adelante" Fred Brooks - "No Silver Bullet - Essence and Accidents of Software Engineering". IEEE Computer,1987.

Partes de la User Story → Está compuesta por tres partes:

1. Tarjeta: es una descripción de la historia, utilizada para planificar y como recordatorio. Tiene una sintaxis particular que es la siguiente:

<<Frase Verbal>>

Como <<nombre del rol>> yo puedo <<actividad>> de forma tal que <<valor de negocio>>

- Nombre del rol: representa quien está realizando la acción o quien recibe el valor de la actividad.
- Actividad: representa la acción que realizará el sistema.
- Valor de negocio: es la más importante y representa el por qué es necesaria la actividad. Esto justifica la razón de que el Product Owner escriba las US. Permite priorizar el desarrollo.
- Frase verbal: no es obligatoria, es una forma corta de referenciar la US. Representa la funcionalidad que se expresa en la User Story.

- **Criterios de aceptación:** son diferentes criterios sobre la actividad que se necesita implementar, que permiten definir los límites de la U.S, facilitan las tareas de desarrolladores y testers para probar la funcionalidad. Se deben redactar independientes a la implementación (alto nivel de abstracción).
- **Pruebas de aceptación:** acompañan a la tarjeta en la parte trasera, y es donde se capturan todos los detalles (que se manifiestan en la conversación) de la User, permitiendo determinar cuándo una historia está completa.

2. Conversación: Es la parte más importante de la US, ya que explicita la comunicación entre el Product Owner y el Equipo de desarrollo que permite compensar la falta de especificación y detalle. El acuerdo por sobre la negociación del contrato y el principio de técnicos y no técnicos trabajando juntos durante todo el proyecto están asociadas a esta parte de la US.

El Product Owner es parte del equipo, por lo cual se espera responsabilidad, compromiso y confianza de su parte para hacerse cargo y llegar a un acuerdo común con el equipo sobre lo definido acerca de la funcionalidad.

También es posible dejar la conversación persistente en grabaciones, minutas o softwares de gestión que posibiliten la conversación. De todas maneras, la conversación se materializa de forma distribuida en la tarjeta y en las pruebas de aceptación.

En simples palabras, son discusiones acerca de la historia que sirven para desarrollar los detalles de la historia.

3. Confirmación: Definición de un acuerdo entre el Product Owner y el Equipo para demostrar que la característica del producto realmente se implementó. Conforma a la definición de las pruebas de aceptación. El PO decide si las pruebas de aceptación son válidas. Pruebas que se usan para determinar cuándo una US está completa.

Ventajas

- Prioriza la comunicación verbal por sobre la escrita (recordar la conversación es la parte más importante).
- Son comprensibles tanto para el cliente/usuario (Product Owner) como para el equipo de desarrollo (recordar que se escriben en términos de negocio).
- Tienen el tamaño justo para permitir planificar.
- Permiten el desarrollo iterativo al dividir todas las funcionalidades del producto en US.
- Permiten desestimar los detalles, hasta que se logre una mejor comprensión sobre cuál es la necesidad concreta de los usuarios.

Desventajas

- En proyectos grandes, es difícil entender la relación entre U.S (son independientes entre sí).
- No son apropiadas para equipos demasiado grandes (por el tema de priorizar la conversación verbal, por sobre la escrita).

El Product Backlog: Una Pila Dinámica y Priorizada

El **Product Backlog** es un contenedor transitorio de requerimientos, organizado como una lista de ítems (PBIs) con sus estimaciones de tamaño.

- **Estructura y Prioridad:** Se organiza por relevancia; en la parte superior se ubican las **User Stories (U.S.) de mayor prioridad** y con una granularidad fina (mucho detalle), mientras que al fondo quedan las de menor prioridad y granularidad gruesa.
- **Gestión del Product Owner:** El P.O. es el único responsable de decidir el orden de la pila según el **valor de negocio**, pudiendo cambiar la priorización o remover ítems en cualquier momento.
- **Naturaleza "Just in Time":** El backlog nunca está completo al 100%. Los requerimientos se descubren y refinan continuamente; se agregan nuevos ítems a medida que surgen necesidades y se quitan los de mayor prioridad para implementarlos en cada iteración según la **velocidad del equipo**.

User Stories como "Porciones Verticales"

Para que una historia de usuario aporte valor real, debe tener un poco de:

- Interfaz
- Lógica de negocio
- Base de datos

Relación con la arquitectura en capas

La User es una porción vertical, la cual abarca todas las capas de arquitectura: capa de datos, lógica de negocio y capa de presentación. Esto permite entregar una característica terminada al cliente.

Estimación y Story Points

El tamaño de las tareas en ambientes ágiles no se mide en tiempo (horas-hombre), sino en **Story Points**, que son unidades de medida **relativas**.

Por qué medidas relativas: Se basa en la premisa de que los seres humanos no son buenos estimando en términos absolutos, pero sí comparando tamaños (por ejemplo, saber que una tarea es el doble de compleja que otra).

- **Factores que determinan el tamaño:** El "peso" de una U.S. se define por tres aspectos fundamentales:
 1. **Complejidad:** Dificultad inherente.
 2. **Esfuerzo:** Hs de trabajo que requerirá la implementación.
 3. **Incertidumbre:** Falta de información, ya sea **técnica** (desconocimiento de una tecnología) o **de negocio** (dudas sobre cómo interactuará el usuario).

Escalas no lineales: el aumento exponencial del riesgo

Vamos a usar la serie de Fibonacci para realizar las estimaciones ya que permite modelar el crecimiento exponencial ya que el riesgo en el sw NO ES LINEAL

Como vamos a hacer las estimaciones → EN EQUIPO y POR CONSENSO llegando a un ACUERDO

- **VELOCIDAD:** comparar A con B requiere una fracción de segundo que estudiar a B aislada
- **COLABORACION:** desplaza el foco del individuo al equipo mejorando drásticamente la dinámica grupal
- **EFICIENCIA:** optimiza drásticamente el tiempo invertido en el análisis de historias de usuario

¿Por qué estimamos?

- Para predecir completitud.
- Para administrar riesgos.

Antes de que el proyecto comience, el líder del proyecto y el equipo de software deben estimar el **trabajo** que habrá de realizarse, los **recursos** que se requerirán y el **tiempo** que transcurrirá desde el principio hasta el final.

Las estimaciones requieren:

- Experiencia.
- Acceso a buena información histórica (métricas).
- El valor para comprometerse con predicciones cuantitativas cuando la información cualitativa es todo lo que existe.

Problemas de estimaciones tradicionales

- Estimar demasiado pronto.
- Resistencia a las reestimaciones.
- Estimaciones que buscan precisión.
- Estimaciones hechas por gente distinta a la que hace el trabajo.
- Estimaciones que pretenden ser absolutas (inflexibles).

Existe una serie de consideraciones importantes:

- Son medidas relativas no absolutas.
- No es una medida basada en el tiempo.
- Las personas no saben estimar en términos absolutos, somos buenos comparando.
- Comparar es generalmente más rápido.
- Se obtiene una mejor dinámica grupal y pensamiento de equipo más que individual.
- Se emplea mejor el tiempo de análisis de las Stories.

¿Qué se estima?

Lo que se estima en ambientes ágiles, es solo el tamaño de las US (o Features) usando como unidad de medida los Story Points, y teniendo en cuenta 3 aspectos de estas: incertidumbre, complejidad y esfuerzo. Como la complejidad de las US tiende a incrementarse exponencialmente, se utilizan comúnmente series numéricas de esta índole, siendo una de las más comunes, la serie de Fibonacci.

Las estimaciones van a ser propias de cada equipo, para un equipo, una US de 8 puntos puede no significar lo mismo que para otro equipo, pero lo que importa es que en un mismo equipo, una US de 5 puntos, siempre sea más chica que una de 8.

La diferencia crítica Tamaño vs Esfuerzo

La complejidad es independiente de quien haga el proyecto, y el esfuerzo depende específicamente de la persona que lo va a llevar a cabo

Diferencia de tamaño con el Esfuerzo: El tamaño es una propiedad de la **tarea**, mientras que el esfuerzo se refiere a las **horas-hombre reales** necesarias para completarla.

Premisa: la descripción de la tarea NO ES el esfuerzo. El tamaño NO ES solo el esfuerzo

TAMANO DE SW = MAGNITUD GENERAL + ESFUERZO FISICO/MENTAL + COMPLEJIDAD ARQUITECTONICA

Tamaño: Es la medida de qué tan **compleja y grande** es una User Story, calculada en función del trabajo requerido y la **incertidumbre** (falta de información) que presenta. Se utiliza como base de comparación entre distintas historias.

- **El factor del "quién":** Una misma US tiene un **tamaño fijo** para el equipo, pero el **esfuerzo varía** según quién la realice, dependiendo de su experiencia, habilidades y familiaridad con el dominio.

Formas de estimar el tamaño

1. Número del 1 al 10.
2. Talles de remeras: S, M, L
3. Serie exponencial de base 2: 2, 4, 8, 16, 32, 64, etc.
4. Serie de Fibonacci: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, etc.

¿Para qué se estima?

1. En un primer momento para determinar cuántas Users se puede comprometer el equipo de trabajo a terminar en una iteración.

2. Al finalizar la iteración, contamos los puntos de historia correspondiente a las Users aceptadas por el Product Owner y obtenemos la velocidad del sprint, la cual es una de las métricas fundamentales en ambientes ágiles.

¿Cómo se estima la duración del proyecto?

Si la estimación se realiza utilizando Story Points para medir la complejidad relativa de las User Stories, para determinar la duración de un proyecto se realiza la derivación tomando el número total de story points de sus US y dividiéndolas por la velocidad del equipo.

Modelado de Roles

Se utilizan las **tarjetas de rol de usuario**, aquí se anotan las características en general que va a tener el usuario.

Técnicas adicionales

1. **Personas:** esta técnica busca ser más específica acerca de la descripción de la persona concreta que va a utilizar el sistema.
2. **Personajes extremos:** que personas en términos extremos podrían llegar a utilizar el software.
3. **Proxies (Usuarios Representantes):** es un representante de todos los usuarios. Debe tener capacidad de tomar decisiones desde el negocio.

Métodos utilizados para estimar

Se recomienda usar varios métodos de estimación y compararlos entre sí, ya que todos pueden requerir un **factor de ajuste** para calibrar mejor el resultado.

1. Basados en la experiencia Se apoyan en información o conocimiento previo para estimar nuevos proyectos.

a. Datos históricos: Consiste en recolectar datos de proyectos anteriores para formar una base de conocimiento útil para futuras estimaciones. La idea es que la experiencia no quede solo en una persona, sino en la organización.

Dentro de estos datos se distinguen:

- **Industry Data:** datos de otras organizaciones o del mercado, útiles para comparaciones generales.
- **Historical Data:** datos internos de la organización sobre proyectos ya finalizados.
- **Project Data:** datos del mismo proyecto, pero de etapas anteriores.

b. Juicio experto: Es uno de los métodos más usados. Se basa en la experiencia de una o varias personas expertas, aunque puede ser riesgoso si se utiliza como única técnica.

- **Juicio experto puro:** un experto analiza las especificaciones y estima según su experiencia. El problema es que la organización depende mucho de esa persona. Puede usarse el método **optimista, pesimista y habitual** para obtener una estimación más equilibrada.
- **Juicio Wideband Delphi:** es una estimación grupal. Varios expertos estiman individualmente, comparan opiniones de forma anónima y repiten el proceso hasta llegar a un acuerdo razonable. Es la base del **Planning Poker**.

2. Analogía Consiste en comparar el proyecto actual con otros proyectos similares, teniendo en cuenta factores como **tamaño, complejidad y cantidad de usuarios**. Puede tener mucho margen de error porque el conocimiento de un proyecto no siempre se puede trasladar a otro.

3. Otros métodos

a. Basados exclusivamente en los recursos: Se estima según el personal disponible y el tiempo con el que se cuenta. El problema es que el trabajo puede expandirse hasta consumir todos los recursos, aunque no se logren los objetivos.

b. Basados en la capacidad del cliente o en el mercado: La estimación se hace según cuánto puede pagar el cliente o lo que permite el mercado, no tanto según el producto en sí. A veces se cobra menos para competir.

c. Basados en los componentes del producto o en el proceso de desarrollo:

- **Bottom-up:** se divide el producto en unidades pequeñas, se estima cada una y luego se suma todo.
- **Top-down:** se divide el producto en grandes bloques y se estima cada componente general.

d. Métodos algorítmicos:

Usan algoritmos con variables como tamaño, complejidad y conocimiento del dominio. Aunque se basan en cálculos matemáticos, no garantizan una estimación exacta.

Poker Planning o Poker Estimation

Es una técnica de estimación en ambientes ágiles publicada por Mike Cohn.

El equipo estima su propio trabajo y los miembros opinan sobre lo mismo, donde se mezcla la experiencia y el conocimiento de cada uno con la posibilidad de compartir con el resto del grupo. Un detalle es que el Product Owner participa en la planificación, pero no realiza estimaciones, suele ser el moderador, pero no es necesario que sea así siempre.

Escala complejidad en el software es exponencial → serie de Fibonacci.

1. **Valor 1 o 1/2:** son los valores más bajos y se aplican a una funcionalidad pequeña.
2. **Valor 2- 3:** funcionalidad pequeña a mediana.
3. **Valor 5:** funcionalidad media.
4. **Valor 8: funcionalidad** grande, donde nos preguntamos si es posible dividir.
5. **Valor 13 o más:** necesariamente hay que dividir ya que no ingresa en el sprint.

Procedimiento

2. Se define una lista de actividades, módulos, casos de uso, Users stories, etc.
3. Se acuerda cuál de las anteriores es la que tiene **menor grado de incertidumbre** y se la define como **canónica** asignándole 1 o 2 story point (es la más simple). Esta User es utilizada como referencia para comparar.
4. Cada miembro elige una carta con el valor de la estimación del tamaño de la U.S. (comparado con la U.S. canónica y teniendo en cuenta los 3 aspectos nombrados) y se espera a que todos decidan.
5. Utilizando **juicio experto**, cada integrante del equipo de desarrollo expone su valor de estimación de forma simultánea, por medio de las cartas, dándolas vuelta a las que estaban boca abajo. En caso de que la estimación difiera, el mayor y el menor estimador justifican sus estimaciones para conocer sus opiniones.
6. En caso de que la primera vuelta existiese diferencias considerables, se realiza nuevamente la ejecución del paso 4.
7. Si las estimaciones no convergen, entonces se llega a una estimación intermedia o promedio.

Consideraciones

- Para realizar una estimación **se debe tener en cuenta el criterio de Done** y no solo la tarea de programación, es decir, todo lo que hay que hacer para terminar una US.
- El tiempo para realizar las estimaciones, como cualquier otra actividad en los ambientes ágiles, es time boxing.
- Las US por estimar son las menos posibles, en orden con el principio de diferir compromisos.

Ventajas

- Permite opiniones multidisciplinarias de todo el equipo, lo cual enriquece la estimación.
- Al generar debates y discusiones, permite lograr una mejor comprensión del dominio
- Estudios demuestran que reunir estimaciones individuales conducen a mejores resultados, que estimar de forma grupal.

Estimaciones en Lean

En Lean, el cual es más extremo que las metodologías ágiles, asume que las estimaciones son opcionales y en algunas ocasiones pueden convertirse en un desperdicio dado que tienen una alta probabilidad de no coincidir con la realidad, dado que cualquier variable que se modifica, modificara a la estimación también

Criterios de aceptación

Es una nota no obligatoria que define un acuerdo respecto a cómo se debe comportar el software para que el PO acepte la implementación, es recomendable que este debido a que sino los desarrolladores no tendrían un parámetro para definir si la misma fue cumplimentada o no. Debe ser escrito en términos objetivos y concretos.

- Definen límites para una User Story (US)
- Ayudan a que los PO respondan lo que necesitan para que la US provea valor (requerimientos funcionales mínimos).
- Ayudan a que el equipo tenga una visión compartida de la US.
- Ayudan a desarrolladores y testers a derivar las pruebas.
- Ayudan a los desarrolladores a saber cuándo parar de agregar funcionalidad en una US.

¿Cuáles son los criterios de aceptación buenos?

- Definen una intención, no una solución. Ejemplo: El usuario debe elegir al menos una cuenta para operar.
- Son independientes de la implementación.
- Relativamente de alto nivel, no es necesario que se escriba cada detalle.

Pruebas de aceptación

Son declaraciones de intención de que hay que probar. Se prueban escenarios exitosos y escenarios que fallen.

El Product Owner acepta la User como implementada si todas las pruebas de aceptación se cumplen.

Aspecto de la Estimación	Ambiente Tradicional (Procesos Definidos)	Ambiente Agile (Procesos Empíricos)
Gestión de Dimensiones	El alcance es fijo y el producto está definido de antemano. El tiempo y los recursos se derivan de este alcance.	El tiempo y los recursos son fijos (iteraciones). El alcance es la variable que se ajusta según la capacidad del equipo.
Precisión	Busca ser lo más precisa posible, ya que es la base para toda la planificación posterior.	La precisión extrema se considera desperdicio (waste) según la filosofía Lean.
Principal Diferencia (Medida)	Las estimaciones son absolutas .	Las estimaciones son relativas ; es más fácil y rápido para las personas comparar tamaños que asignar valores exactos.
Quién estima	La realiza el líder del proyecto o un experto externo, lo que suele generar desvíos respecto al trabajo real.	La realiza el equipo de desarrollo (quienes harán el trabajo) de forma colectiva, disminuyendo desvíos.
Naturaleza del Compromiso	Se asumen como promesas o compromisos fijos (planificaciones), lo cual es un error común.	Se consideran probabilidades . Una falla no extiende el tiempo, sino que obliga a reajustar el alcance.
Momentos para Estimar	Al comienzo del proyecto , luego de los requerimientos y el diseño, o cuando cambia el alcance.	De manera recurrente al comienzo de cada sprint .
Incertidumbre y Certeza	Al inicio, la falta de información puede causar errores de estimación de hasta un 400% .	La incertidumbre se gestiona mediante el refinamiento continuo y la visión compartida del producto.

Visión de Producto: En Agile se estima sobre una **visión**, en lo tradicional se estima sobre un **producto definido**.

Pensamiento de Equipo: Estimar por comparación en ambientes ágiles fomenta la **dinámica grupal** por encima de la opinión individual.

Causa de errores: En lo tradicional, el hecho de que el líder sea quien estima sin considerar la capacidad real de cada ejecutor es una de las causas principales de fracaso.

Spikes

Son un tipo especial de US que **se producen por la incertidumbre no puede ser** estimada y por lo tanto no cumple con la definición de listo, Una vez resuelta la incertidumbre, la Spike se convierte en una o más US. Es una característica más del producto a la cual el equipo le tiene que dedicar tiempo para:

5. Familiarizarse con una nueva tecnología o dominio.
6. Investigar y prototipar para ganar confianza frente a:
 - a. Riesgos tecnológicos.
 - b. Riesgos funcionales

Planificación

En ambientes ágiles, es necesario que quien planifica sea capaz de analizar el contexto del proyecto cada cierto período y hacer aquellos ajustes que sean necesarios

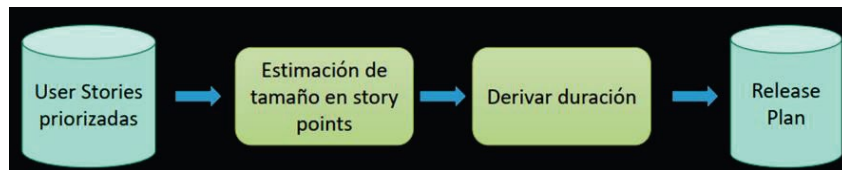
“Planning is everything. Plans are nothing” → esto apunta a que lo importante de planificar, es el proceso en sí. No el hecho de realizar una documentación exhaustiva que respete determinados templates

Los equipos ágiles afrontan esta situación, planificando al menos en 3 niveles:

Planificación de Release

Este proceso se da al inicio del proyecto, donde se analiza y determina qué US, épicas o Temas serán desarrollados en una nueva entrega (Release) del producto. Este plan se va actualizando mientras el proyecto transcurre (usualmente al comienzo de cada iteración), de manera que siempre refleja las expectativas actuales sobre qué se incluye en la entrega.

Una vez que se priorizan las US a implementar, se estima su tamaño con Story points, y teniendo en cuenta la velocidad de trabajo del equipo, se deriva la duración de la Release o del proyecto (la cantidad de sprints necesarios para obtener el release).



Planificación de Iteración

Este proceso se da al comienzo de cada iteración, donde el PO identifica aquellas US de mayor prioridad que deberán ser implementadas en la iteración. Esto se hace teniendo en cuenta el tiempo y la velocidad de trabajo del equipo.

Planificación del día

Este proceso permite coordinar y sincronizar el trabajo del equipo, normalmente en reuniones diarias. Allí sólo se discuten las tareas a realizar durante el día, haciendo foco en las actividades que deberán ser desarrolladas para lograr cumplir con una tarea específica.

Scrum

Scrum es un marco de trabajo liviano que permite generar valor mediante soluciones adaptables para problemas complejos. Se basa en un enfoque **iterativo e incremental**, donde el trabajo se organiza en ciclos llamados **Sprints**.

En Scrum, el **Product Owner** ordena el trabajo en el **Product Backlog**, el **Scrum Team** desarrolla una parte de ese trabajo durante el Sprint y luego se inspeccionan los resultados para adaptarse en el siguiente ciclo.

Scrum no da instrucciones detalladas, sino que define reglas, roles, eventos y artefactos mínimos. Se basa en el **empirismo** y el pensamiento **Lean**, es decir, aprender desde la experiencia, tomar decisiones según lo observado, reducir desperdicios y enfocarse en lo esencial.

Pilares de Scrum

Transparencia: el proceso, el trabajo y los artefactos deben ser visibles y comprensibles para quienes trabajan y para quienes reciben el producto. Sin transparencia, las decisiones pueden aumentar el riesgo.

Inspección: el progreso y los artefactos deben revisarse frecuentemente para detectar problemas o desvíos.

Adaptación: si algo se desvía o el producto no cumple lo esperado, el equipo debe ajustar el proceso o el trabajo lo antes posible.

Valores de Scrum

Compromiso, enfoque, apertura, respeto y coraje.

Scrum Team → El **Scrum Team** es la unidad principal de Scrum. Está formado por:

Product Owner: responsable de maximizar el valor del producto. Gestiona el **Product Backlog**, define y comunica el **Objetivo del Producto**, ordena los ítems pendientes y se asegura de que el backlog sea claro, visible y comprendido.

Es una sola persona, no un comité, aunque representa las necesidades de muchos interesados.

Desarrolladores: quienes crean el incremento funcional del producto. Sus responsabilidades principales son:

- Crear el **Sprint Backlog**.
- Cumplir con la **Definición de Hecho**.
- Adaptar el plan diariamente.
- Responsabilizarse mutuamente como profesionales.

Scrum Master: ayuda a que Scrum se entienda y se aplique correctamente. Sirve al equipo, al Product Owner y a la organización.

Sus funciones principales son:

- Facilitar la autogestión del equipo.
- Ayudar a crear incrementos de valor.
- Eliminar impedimentos.
- Asegurar que los eventos Scrum se realicen correctamente.
- Capacitar a la organización en Scrum.

Eventos de Scrum → Los eventos permiten generar regularidad, inspección y adaptación.

Sprint

El **Sprint** es el evento contenedor de Scrum. Tiene una duración fija de un mes o menos. Durante el Sprint se transforma el trabajo seleccionado en un incremento de valor.

Dentro del Sprint no se deben hacer cambios que pongan en riesgo el objetivo, la calidad no debe disminuir y el alcance puede renegociarse con el Product Owner si se aprende algo nuevo.

Sprint Planning

Es la reunión que inicia el Sprint. Allí el equipo define:

¿Por qué este Sprint es valioso?

Se establece el **Sprint Goal**.

¿Qué se puede hacer este Sprint?

Se seleccionan ítems del Product Backlog.

¿Cómo se realizará el trabajo?

Los desarrolladores planifican las tareas necesarias.

El resultado es el **Sprint Backlog**.

Daily Scrum

Es una reunión diaria de máximo 15 minutos donde los desarrolladores inspeccionan el progreso hacia el Sprint Goal y adaptan el plan del día.

Sirve para mejorar la comunicación, detectar impedimentos y tomar decisiones rápidas.

Sprint Review

Se realiza al final del Sprint. El equipo muestra el incremento a los stakeholders y se inspecciona el resultado obtenido.

No debe ser solo una presentación, sino una sesión de trabajo para recibir retroalimentación y adaptar el Product Backlog.

Sprint Retrospective

Es la última reunión del Sprint. El equipo analiza cómo trabajó: personas, interacciones, procesos, herramientas y calidad.

El objetivo es identificar mejoras para aumentar la eficacia y la calidad en los próximos Sprints.

Refinamiento del Product Backlog

Es una actividad continua donde se detallan, dividen y aclaran los ítems del Product Backlog para que estén listos para futuros Sprints.

Artefactos de Scrum → Los artefactos representan trabajo o valor y buscan maximizar la transparencia.

Product Backlog

Es la lista ordenada de todo lo necesario para mejorar el producto. Es gestionado por el Product Owner.

Su compromiso asociado es el **Product Goal**, que representa el objetivo a largo plazo del producto.

Sprint Backlog Es el plan del Sprint. Incluye:

- El **Sprint Goal**.
- Los ítems seleccionados del Product Backlog.
- El plan para construir el incremento.

Es creado y actualizado por los desarrolladores.

Incremento

Es una parte funcional y utilizable del producto. Cada incremento se suma a los anteriores y debe cumplir con la **Definición de Hecho**.

Definición de Hecho

La **Definition of Done** indica cuándo un trabajo puede considerarse realmente terminado. Si un ítem no cumple esta definición, no puede presentarse como parte del incremento.

Definición de Listo

La **Definition of Ready** indica cuándo una User Story está suficientemente clara y preparada para entrar a un Sprint.

La **Definition of Done**, en cambio, indica cuándo esa historia está correctamente implementada y puede ser revisada o puesta en producción.

Timebox

El **timebox** establece límites de tiempo para las ceremonias y actividades. Esto evita perder tiempo innecesario y obliga a negociar el alcance, no el tiempo ni los costos.

Si no se llega al final del Sprint, no se extiende el tiempo: se entrega menos alcance.

Capacidad del equipo en un Sprint

La capacidad indica cuánto trabajo puede comprometer el equipo en un Sprint. Se estima durante el Sprint Planning y puede medirse en **Story Points** o en **horas ideales**, según la madurez del equipo.

Niveles de planificación → Scrum puede planificarse en distintos niveles:

Portfolio: planificación estratégica de uno o más productos o proyectos.

Producto: Es el que trabaja el QUE, que voy a desarrollar, que construyo, que funcionalidad tiene, hasta donde llega y que incluye

Release: planificación de entregas.

Iteración: es como voy a trabajar los reléase - definición del Sprint Goal y Sprint Backlog.

Día: seguimiento diario del progreso.

Responde a estas preguntas

Portafolio - en que invertir

Release – Que y Cuando entregar

Producto - cómo evoluciona

Sprint - que hacemos ahora

Daily - como avanzamos hoy

Planificación en Scrum

Planificación de producto

La **planificación de producto** traduce la estrategia en una visión concreta del producto.

Define qué producto se quiere construir, qué valor va a generar, para quién está pensado, qué funcionalidades principales tendrá y con qué recursos o restricciones se cuenta.

Participan el **Product Owner**, los **stakeholders** y el **Scrum Team**. A partir de una idea inicial se definen elementos como:

- **Visión del producto**
- **Product Backlog priorizado**
- **Roadmap del producto**
- Recursos disponibles
- Restricciones del producto y del proyecto

El objetivo es organizar las entregas futuras y orientar el desarrollo hacia una solución concreta que genere valor al usuario.

Product Backlog y versiones

El **Product Backlog** puede contener distintos tipos de elementos, como **épicas, User Stories, temas, spikes**, mejoras o correcciones.

Ese listado se va dividiendo en **releases**, que son versiones del producto. Las versiones pueden numerarse, por ejemplo:

- **Versión 1.0:** primera versión importante.
 - **Versión 1.1 o 1.2:** correcciones o mejoras menores.
 - Se cambia el primer número cuando hay un cambio importante en el producto.
-

Planificación de release

La **planificación de release** define qué funcionalidades se van a entregar en un período determinado.

Sus principales entradas son:

- **Product Backlog**
- **Visión del producto**
- **Velocidad del equipo**
- Restricciones existentes

Su salida principal es el **plan de release**, que indica qué funcionalidades se entregarán y cuándo.

Diferencia entre planificación de producto y planificación de release

La **planificación de producto** es más general y estratégica. Define la visión, el objetivo del producto, el roadmap y los recursos necesarios. Participan el Product Owner, stakeholders, Scrum Team y otros interesados, como proveedores o áreas de apoyo.

La **planificación de release** es más concreta. Se enfoca en decidir qué funcionalidades del Product Backlog se incluirán en una entrega específica y en qué momento se liberarán. Participan principalmente el Product Owner y el Scrum Team.

Producto: define hacia dónde va el producto.

Release: define qué se entrega en una versión concreta.

Cadencia de Sprint

La **cadencia de Sprint** indica cada cuánto tiempo el equipo entrega valor. Mientras más frecuente sea la entrega, mayor será el feedback y mayor la capacidad de adaptación.

Una release puede ocurrir:

- Después de múltiples Sprints.
- Después de cada Sprint.
- Después de una feature o User Story específica.

La duración del Sprint puede definirse según el contexto, pero conviene mantenerla estable para dar ritmo y previsibilidad al equipo.

Planificación de iteración o Sprint Planning

La **planificación de iteración** es la primera actividad de cada Sprint. En esta reunión se decide qué User Stories se van a trabajar, cuántas puede asumir el equipo y cómo se organizará el trabajo.

También se tienen en cuenta:

- Capacidad del equipo.
- Disponibilidad de las personas.
- Dependencias técnicas o externas.
- Prioridades del Product Backlog.
- Refinamiento de las funcionalidades.

Por ejemplo, si alguien del equipo no estará disponible o si una funcionalidad depende de una integración externa, eso se considera antes de comprometer trabajo para el Sprint.

Planificación diaria o Daily Scrum

La **Daily Scrum** es una reunión diaria de máximo 15 minutos. Sirve para inspeccionar el progreso hacia el objetivo del Sprint y ajustar el plan del día.

Permite:

- Detectar bloqueos.
- Coordinar el trabajo.
- Revisar avances.
- Adaptar el plan rápidamente.

Herramientas de Scrum

Tarjeta de tarea → Representa una tarea del Sprint. Suele incluir código del ítem, nombre de la actividad, responsable, día del Sprint y estimación restante.

Tablero → Permite visualizar el avance del trabajo. Puede dividirse en: **Story, To Do, Work in Process, To Verify y Done.**

Gráficos del Backlog → Sirven para visualizar el progreso del Sprint, release o producto.

Sprint Burn-Down Chart: muestra cuánto trabajo falta.

Sprint Burn-Up Chart: muestra cuánto trabajo se completó.

Combined Burn Chart: muestra trabajo realizado y pendiente.

Frameworks para escalar Scrum

Cuando un producto es demasiado grande para un solo equipo, se utilizan frameworks de escalado. Algunos son **Nexus, LeSS, SAFe y Scrum a escala.**

1.Framework Nexus

Nexus permite coordinar entre tres y nueve equipos Scrum que trabajan sobre un mismo producto, un único **Product Backlog** y un solo **Product Owner**.

Su objetivo es producir un **Integrated Increment**, es decir, un incremento integrado y terminado al menos una vez por Sprint.

Responsabilidades en Nexus

- **Nexus Integration Team:** asegura que se produzca un incremento integrado.
- **Product Owner:** maximiza el valor del producto y gestiona el Product Backlog común.
- **Scrum Master:** asegura que Nexus se entienda y se aplique correctamente.
- **Miembros del Nexus Integration Team:** ayudan a mejorar herramientas y prácticas de integración.

Eventos de Nexus

- **Nexus Sprint:** todos los equipos trabajan para producir un incremento integrado.
- **Refinamiento entre equipos:** permite detectar, reducir o eliminar dependencias.
- **Nexus Sprint Planning:** coordina el trabajo de todos los equipos para el Sprint.
- **Nexus Daily Scrum:** representantes de los equipos revisan problemas de integración.
- **Nexus Sprint Review:** reemplaza las reviews individuales y revisa el incremento integrado.
- **Nexus Sprint Retrospective:** busca mejorar la calidad y eficacia de todo el Nexus.

Artefactos de Nexus

Product Backlog: uno solo para todos los equipos.

Nexus Sprint Backlog: muestra el trabajo de los equipos, dependencias y objetivo común.

Integrated Increment: suma del trabajo integrado de todos los equipos, cumpliendo la Definition of Done.

Framework LeSS

LeSS permite escalar Scrum para productos desarrollados por dos a ocho equipos. Mantiene:

- Un solo **Product Owner**.
- Un único **Product Backlog**.
- Un Sprint común.
- Un incremento integrado del producto.

Para más de ocho equipos se utiliza **LeSS Huge**.

Principios de LeSS → LeSS se basa en Scrum a gran escala, pensamiento sistémico, pensamiento Lean, control empírico, transparencia, mejora continua, foco en el consumidor y visión del producto completo.

Miembros en LeSS

- **Scrum Master:** enseña Scrum y LeSS, ayuda a la organización y puede servir a uno, dos o tres equipos.
- **Product Owner:** gestiona el producto y prioriza el único Product Backlog.
- **Equipos:** son autogestionados y multifuncionales. Trabajan coordinados para entregar un único incremento integrado.

Eventos de LeSS

Sprint Planning One: se seleccionan los ítems del Product Backlog para el Sprint.

Sprint Planning Two: cada equipo define cómo realizará el trabajo.

Daily Scrum: cada equipo realiza su propia reunión diaria.

Sprint Review: es común para todos los equipos y se inspecciona el producto completo.

Retrospective: cada equipo realiza su propia retrospectiva.

Overall Retrospective: analiza problemas entre equipos, organizacionales o sistémicos.

Product Backlog Refinement: todos los equipos refinan ítems para futuros Sprints.

LeSS Sprint: todos los equipos comienzan y terminan el Sprint al mismo tiempo.

Artefactos de LeSS

Incremento potencialmente entregable: resultado integrado de todos los equipos.

Product Backlog: único para todos los equipos.

Sprint Backlog: cada equipo tiene el suyo propio.

También existe una **Definition of Done común**, aunque cada equipo puede agregar criterios más estrictos.

LeSS Huge Se utiliza cuando hay más de ocho equipos. Mantiene algunos elementos de LeSS, como:

- Un Product Backlog general.
- Un DoD común. (definition of done)
- Un DoR común. (definition of ready)
- Un Sprint común.
- Un incremento potencialmente entregable.

La diferencia es que incorpora estructuras por áreas, como **Area Product Manager, Area Product Owners, Area Product Backlogs y Area Product Vision**, para organizar mejor el trabajo a gran escala.

¿Qué es una métrica?

Una **métrica** es un número que permite medir objetivamente un aspecto del **proceso, proyecto o producto**. No alcanza con usar escalas como “bueno” o “muy bueno”; hay que poder cuantificar, por ejemplo, cuántos defectos hubo, cuántas historias se terminaron o cuánto esfuerzo se usó.

Medir siempre tiene un costo, porque requiere tiempo, recursos y análisis. Por eso no conviene medir “por medir”, sino solo aquello que aporte valor y ayude a tomar decisiones.

¿Para qué medir?

- Controlar y supervisar el avance.
- Estimar proyectos futuros.
- Evaluar costos, tiempos, esfuerzo o calidad.
- Mejorar el proceso, el proyecto o el producto.

Dominios de métricas

Métricas de proceso

Miden el proceso a nivel organizacional. Buscan mejorar la forma de trabajar de la organización a largo plazo.

- Defectos promedio por proyecto.
- Desviación organizacional de estimaciones.
- Tiempos promedio de planificación.
- Errores detectados antes de releases.

Son métricas más usadas en enfoques tradicionales, porque se asume que la experiencia de un proyecto puede extrapolarse a otros.

Métricas de proyecto

Miden recursos del proyecto, como tiempo, costo, esfuerzo y planificación. Sirven para controlar el desarrollo y hacer ajustes durante el proyecto.

- Costos estimados vs. reales.
- Fechas planificadas vs. reales.
- Esfuerzo por tarea.
- Cantidad de cambios.

Métricas de producto

Miden lo que se está construyendo. Se enfocan en calidad, defectos, estructura del código, casos de prueba y cumplimiento de requerimientos.

- Cantidad de defectos.
 - Defectos por severidad.
 - Casos de prueba.
 - Complejidad del código.
 - Cantidad de historias terminadas.
-

Métricas en el enfoque tradicional

En el enfoque tradicional se miden los tres dominios: **proceso, proyecto y producto**. Hay más métricas y suelen definirse desde la planificación del proyecto.

Las métricas dependen del rol:

- A los **testers** les interesan defectos y esfuerzo.
- Al **líder de proyecto** le interesan costos, calendario, plazos y esfuerzo.
- A la organización le interesan métricas generales para mejorar procesos.

Las métricas básicas de un proyecto de software son:

- **Tamaño del producto**
- **Esfuerzo**
- **Calendario**
- **Defectos**

Estas se relacionan con la triple restricción: **alcance, tiempo y esfuerzo/costo**.

Métricas en ambientes ágiles

En ágil, las métricas deben servirle al equipo y **no son extrapolables automáticamente a otros equipos o proyectos**. Esto se debe a que cada equipo tiene su propio contexto, forma de trabajo, experiencia y ritmo.

El principio ágil más importante relacionado con métricas es:

“La mejor métrica de progreso es el software funcionando.”

Esto significa que en ágil se priorizan métricas vinculadas al **producto real entregado**, no a documentación o controles excesivos. Se mide solo lo necesario y lo que aporta valor.

Métricas ágiles principales

1. Velocidad

La **velocidad** es la métrica más importante en Scrum. Mide la cantidad de **Story Points** completados en un Sprint y aceptados por el **Product Owner**. Importante: la velocidad **se calcula al final del Sprint**, no se estima antes.

No se cuentan historias parcialmente terminadas. Solo se cuentan las historias terminadas, funcionando y aceptadas.

Sirve para:

- Conocer el ritmo real del equipo.
- Analizar la estabilidad del equipo.
- Planificar futuros Sprints.
- Ver si el equipo mantiene un desarrollo sostenible.

Ejemplo: Si el equipo terminó y el PO aceptó historias por 25 P en un Sprint, la vel. de ese Sprint fue **25 Story Points**.

2. Capacidad

La **capacidad** mide cuánto trabajo puede comprometer el equipo para un Sprint determinado.

Se estima al comienzo del Sprint y puede medirse en:

- **Horas ideales**
- **Story Points**

Sirve para planificar cuántas historias del Product Backlog se pueden tomar para el Sprint.

A diferencia de la velocidad, que se calcula con lo realmente terminado, la capacidad es una estimación previa de cuánto trabajo puede asumir el equipo.

3. Running Tested Features — RTF

La métrica **Running Tested Features** mide la cantidad de funcionalidades que están:

- Terminadas.
- Probadas.
- Funcionando.
- En ejecución.

Sirve para saber cuántas piezas reales del producto ya están operativas.

Su desventaja es que no considera la complejidad. Por ejemplo, puede contar dos historias como “2 funcionalidades funcionando”, aunque una haya sido muy simple y la otra muy compleja.

Si la curva de RTF se mantiene plana o baja, indica un problema: el equipo no está agregando nuevas funcionalidades funcionando o algunas dejaron de ser válidas.

4. Métricas de defectos

Aunque ágil prioriza software funcionando, también necesita medir defectos para controlar la calidad.

Algunas métricas importantes son:

- **Defectos por severidad:** cantidad de defectos clasificados según gravedad.
- **Densidad de defectos:** cantidad de defectos encontrados por historia de usuario o funcionalidad.
- **Cobertura:** grado en que los defectos o pruebas cubren el producto.

Estas métricas ayudan a saber si el producto mantiene calidad mientras evoluciona.

Métricas en Lean / Kanban

En Kanban, el foco está más en el **proceso y el flujo de trabajo**, porque no necesariamente hay un proyecto con inicio y fin, sino un flujo continuo de tareas.

Lead Time

- Es la métrica más importante desde la mirada del cliente.
- Mide el tiempo desde que el cliente pide algo, es decir, desde que entra al Backlog, hasta que se entrega.
- **Lead Time = pedido del cliente → entrega final**
- Sirve para saber cuánto espera realmente el cliente.

Cycle Time

- Mide el tiempo desde que el equipo empieza a trabajar en una funcionalidad hasta que la entrega.
- No incluye el tiempo que la tarea estuvo esperando en el Backlog.
- **Cycle Time = inicio del trabajo → entrega**
- Es una métrica interna del equipo.

Touch Time

- Mide el tiempo en que efectivamente se está trabajando sobre una unidad de trabajo.
- No cuenta tiempos de espera, bloqueos o acumulaciones.
- Sirve para distinguir cuánto tiempo se trabaja realmente y cuánto tiempo se pierde esperando.

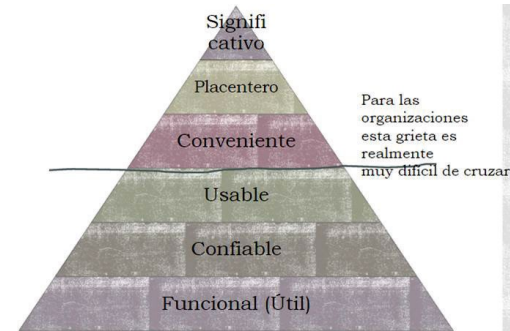
Eficiencia del ciclo de proceso

- Mide qué tan eficiente es el proceso comparando el tiempo efectivo de trabajo con el tiempo total de entrega.
- **Eficiencia = Touch Time / Lead Time**
- Mientras más cercano a **1**, más eficiente es el proceso, porque significa que hubo pocos tiempos de espera.

Gestión de productos de software – Planificación de productos – herramientas para definición de productos de SW

Producto de software

Un **producto de software** es un artefacto construido para satisfacer una necesidad específica a partir de ciertos requerimientos. Al desarrollarlo, es clave enfocarse en lo que realmente aporta valor al cliente.



Planificación ágil del producto

En metodologías ágiles, la visión del producto se representa en el **Product Backlog**, que contiene las funcionalidades identificadas hasta el momento. Esta visión se divide en **releases**, y cada release se construye mediante sucesivas **iteraciones**.

Valor vs Desperdicio → La productividad no se mide por cuánto se construye, sino por si se construye **lo correcto**, es decir, aquello que agrega valor. Si algo no aporta valor, se considera **desperdicio**.

Mínimo Producto Viable — MVP

El **MVP** es una versión inicial del producto que permite obtener la mayor cantidad de aprendizaje validado sobre los clientes con el menor esfuerzo posible. La idea es observar qué hacen realmente los usuarios con el producto, ya que eso es más confiable que preguntarles qué harían.

Un MVP debe:

- Tener valor suficiente para que las personas lo usen o compren.
- Mostrar beneficios futuros para retener usuarios.
- Generar retroalimentación para guiar el desarrollo.

Errores comunes:

- Confundir MVP con MMF o MMP.
- Enfocarse solo en “mínimo” y olvidarse de que debe ser viable.
- Tratar al MVP como producto final y no mejorarlo con el feedback recibido.

El MVP permite validar dos hipótesis principales:

Hipótesis de valor: verifica si el producto realmente entrega valor al cliente.

Hipótesis de crecimiento: verifica cómo nuevos usuarios descubren y adoptan el producto.

Preparar un MVP



Design Thinking

El **Design Thinking** es una metodología centrada en el usuario, usada para crear servicios digitales que respondan a sus necesidades reales. Combina pensamiento **divergente** —generar muchas ideas— y **convergente** —seleccionar la mejor solución—.

Sus etapas son:

- **Empatizar:** comprender las necesidades del usuario.
- **Definir:** identificar qué es realmente valioso para el cliente.
- **Idear:** proponer muchas soluciones posibles.
- **Prototipar:** materializar las ideas seleccionadas.
- **Probar:** validar los prototipos con usuarios y obtener retroalimentación.

Es un proceso iterativo, por lo que se puede volver a etapas anteriores cuando sea necesario.

Lean UX

Lean UX se basa en el ciclo **Build–Measure–Learn**: construir, medir y aprender. Busca validar ideas con usuarios reales lo antes posible para poder corregir o pivotar cuando todavía es barato hacerlo.

La idea central es que el éxito no está en entregar un producto, sino en entregar algo que el cliente realmente use. Por eso se construyen MVP o prototipos tempranos para aprender de la reacción real de los usuarios.

MVP, MVF, MMF, MMP y MMR

MVP — Producto Mínimo Viable: producto inicial creado para validar una hipótesis y obtener aprendizaje con bajo esfuerzo.

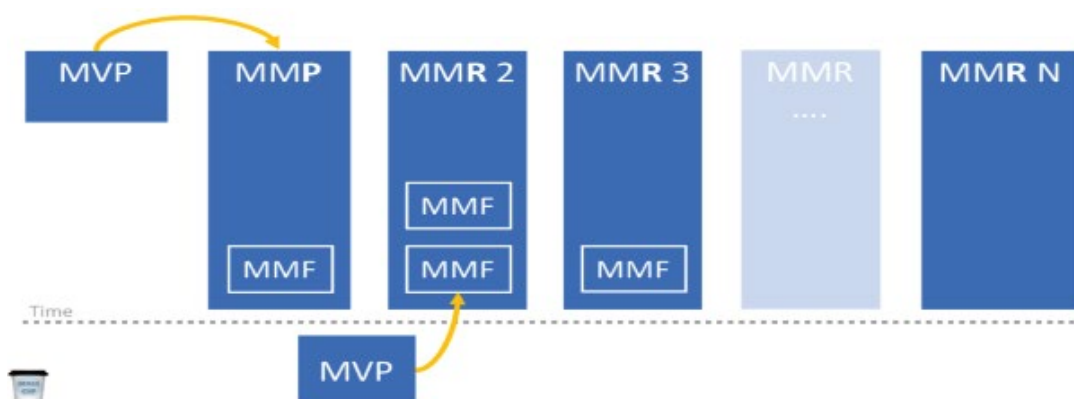
MVF — Característica Mínima Viable: versión pequeña del MVP, pero aplicada a una funcionalidad específica. Sirve para probar si una característica aporta valor real.

MMF — Característica Mínima Comerciable: funcionalidad pequeña que ya puede liberarse porque aporta valor a usuarios y organización.

MMP — Producto Mínimo Comerciable: primer reléase de un MMR orientado a los primeros usuarios, con las características clave para satisfacerlos.

MMR — Release Mínimo Comerciable: release con el conjunto mínimo de características que entrega valor nuevo a los usuarios.

En resumen: el **MVP y el MVF** se enfocan en aprender y validar hipótesis; el **MMF, MMP y MMR** se enfocan más en entregar valor comercializable.



Unidad 3: Gestión de Configuración de SW o Gestión Software como producto

Los sistemas de software cambian constantemente durante su desarrollo y uso. Por eso se generan distintas versiones que deben mantenerse, controlarse y gestionarse para no perder la trazabilidad de los cambios.

Definición de Gestión de Configuración

La **Gestión de Configuración del Software** es una disciplina de soporte que busca mantener la integridad del producto durante todo su ciclo de vida.

- Identificar características técnicas y funcionales de ítems de configuración;
- Documentar características técnicas y funcionales de ítems de configuración;
- Controlar los cambios de esas características identificadas y documentadas;
- Registrar y reportar estos cambios;
- Verificar correspondencia con los requerimientos (trazabilidad).

Sirve para evitar problemas como:

- Pérdida de componentes.
- Pérdida de cambios.
- Regresión de fallas.
- Doble mantenimiento.
- Superposición de cambios.
- Cambios no validados.

La idea principal es que el producto sea íntegro, confiable y trazable.

Ítem de Configuración

Un **ítem de configuración** es cualquier artefacto del producto o proyecto que pueden cambiar o que necesiten ser compartidos entre los miembros del equipo y sobre los cuales necesitamos conocer su estado y evolución.

Puede ser código fuente, documentos, casos de prueba, prototipos, manuales, arquitectura, base de datos, etc.

Se clasifican en:

- **Ítems de producto:** duran mientras exista el producto, por ejemplo código, arquitectura o casos de prueba.
- **Ítems de proyecto:** duran mientras exista el proyecto, por ejemplo plan de proyecto o listado de defectos.
- **Ítems de iteración:** duran una iteración, por ejemplo plan de iteración o Burn-Down Chart.

Versión, variante y configuración

Versión: estado particular de un ítem de configuración en un momento determinado.

Variante: versión alternativa de un IC que evoluciona por separado, por ejemplo para distintos sistemas operativos o plataformas, o entornos o BDD

Configuración: conjunto de todos los ítems de configuración con sus versiones específicas en un momento determinado. Es como una “foto” del producto en cierto punto del tiempo.

Repositorio

El **repositorio** es el lugar donde se almacenan los ítems de configuración, sus versiones, historia, atributos y relaciones. Pueden ser:

Centralizado: un servidor contiene todos los archivos y versiones. Es más fácil de controlar, pero si falla el servidor, se afecta todo.

Descentralizado: cada cliente tiene una copia completa del repositorio. Es más flexible, aunque más complejo de controlar.

Funcionamiento de un repo → Release, Branch y Merge

Release: entrega del sistema liberada para uso del cliente o usuarios.

Branch: rama que permite bifurcar el desarrollo, por ejemplo para experimentar, corregir errores o adaptar el software a distintas plataformas.

Merge: integración de ramas. Se recomienda mantener la rama principal como versión estable e integrar allí los cambios aprobados.

Línea base

Una **línea base** es un conjunto de ítems de configuración revisados y aprobados formalmente. Sirve como punto de referencia para desarrollos futuros.

Sirve para:

- Saber cuál era la última versión estable.
- Hacer rollback.
- Saber qué se puso en producción.
- Reproducir el entorno de desarrollo en un momento dado.

Tipos:

Operacional: contiene código ejecutable aprobado y listo para usar.

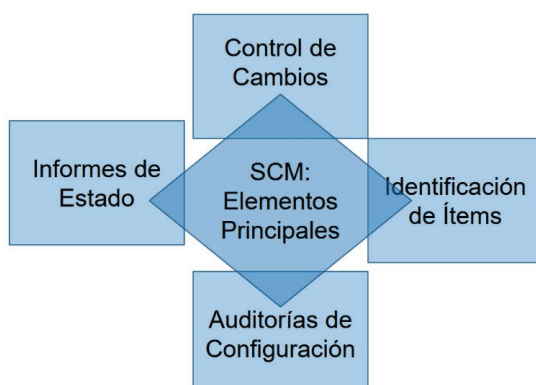
De especificación o documentación: incluye documentos como requerimientos o diseño, antes de tener código.

Planificación de la gestión de configuración

El plan de gestión de configuración debe definirse al inicio del proyecto. Incluye:

- Reglas de nombrado.
- Herramientas a utilizar.
- Estructura del repositorio.
- Roles y comité de control de cambios.
- Procedimientos de cambio.
- Auditorías.
- Registros que deben mantenerse.
- Tipos de documentos administrados.

Actividades relacionadas a la gestión de configuración



1. Identificación de ítems de configuración

- Identificación unívoca de cada ítem de configuración → La lista de ítems de configuración identificados es una lista VIVA puede sufrir modificaciones o variaciones
- Convenciones y reglas de nombrado
- Definición de la Estructura del Repositorio
- Ubicación de los ítems dentro de la estructura del repositorio



Podemos clasificar los IC:

- Ítems de producto: tienen el ciclo de vida más largo, y se mantienen mientras el producto exista.
- Ítems de proyecto: el plan de proyecto, el listado de defectos encontrados, tienen un ciclo de vida de proyecto. Un plan de iteración, un Burn-Down Chart, se mantiene durante una iteración.
- Ítems de iteración: Conocer el ciclo de vida de un ítem permite establecer la nomenclatura de este.

QUE VA A SER UN ITEM / COMO LO VAMOS A NOMBRAR / DONDE LO VAMOS A GUARDAR

2. Control de Cambios

Proceso de control de cambios → el control se hace sobre los ítems de configuración que pertenecen a la línea base, ya que todos los trabajadores del software tienen a dichos ítems como referencia.

El comité de control de cambios está predefinido

Está formado por representantes de todas las áreas involucradas en el desarrollo:

- Análisis, Diseño
- Implementación
- Testing

Etapas del proceso de control de cambios:

- 1) **Recepción de la propuesta:** se recibe una solicitud de cambio sobre una **línea base** ya definida, no sobre un ítem aislado.
- 2) **Análisis de impacto:** el comité evalúa el esfuerzo técnico, los recursos necesarios, los posibles efectos secundarios y el impacto sobre la funcionalidad y la arquitectura.
- 3) **Aprobación y orden de cambio:** si el cambio se autoriza, se genera una orden donde se define qué se hará, bajo qué restricciones y con qué criterios se revisará.
- 4) **Revisión del cambio:** una vez implementado, el comité revisa la modificación y, si corresponde, aprueba una nueva línea base.
- 5) **Notificación:** se informa a los interesados sobre los cambios realizados.

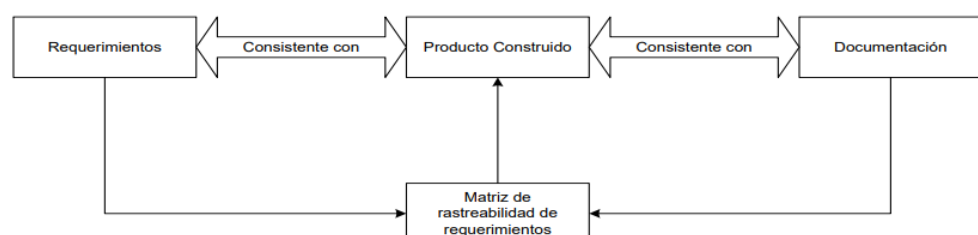
Este proceso permite mantener la **trazabilidad**, es decir, saber qué ítems de configuración se ven afectados por cada cambio.

3. Auditorías de Configuración

Son controles realizados sobre una línea base para analizar el estado del software y verificar si se cumplen las pautas definidas en el plan de gestión de configuración.

Auditoría Funcional de Configuración

Auditoría Física de Configuración



La realiza un auditor externo al equipo, que debe ser:

Objetivo: tiene independencia de criterio y no está condicionado por lo auditado.

Independiente: no depende jerárquica, funcional ni salarialmente del equipo auditado.

La auditoría se hace mientras el producto se está construyendo, con el objetivo de asegurar su **calidad e integridad**. Busca prevenir problemas antes de que sea más costoso corregirlos.

Además, complementa a la revisión técnica, ya que controla si las tareas se están realizando según lo planificado. Para poder hacerla, debe existir un **plan de gestión de configuración** y una **línea base a auditar**.

Trazabilidad o rastreabilidad de requerimientos

Es la relación que se establece entre los distintos ítems de configuración vinculados a un requerimiento, como modelos de análisis, diseño, código y casos de prueba.

Sirve para saber qué partes del sistema se ven afectadas cuando cambia un requerimiento, permitiendo hacer un análisis de impacto.

Además, debe ser bidireccional:

- Desde el requerimiento hacia el código.
- Desde el código hacia el requerimiento.

Sirve a dos procesos básicos: la validación y la verificación

VERIFICACIÓN → construcción del producto CORRECTAMENTE → auditorías físicas

VALIDACIÓN → construcción del producto CORRECTO → auditorías funcionales

Tipos de auditorías

Auditoría física de configuración (PCA) – hace verificación

Asegura que lo que está indicado para cada ICS en la línea base o en la actualización se ha alcanzado realmente.

Auditoría funcional de configuración (FCA) – hace validación

Evaluación independiente de los productos de software, controlando que la funcionalidad y performance reales de cada ítem de configuración sean consistentes con la especificación de requerimientos. La validamos a partir de la ERS

4. Registro e Informe de Estado

- Se ocupa de mantener los registros de la evolución del sistema.
- Maneja mucha información y salidas por lo que se suele implementar dentro de procesos automáticos.
- Incluye reportes de rastreabilidad de todos los cambios realizados a las líneas base durante el ciclo de vida.
- El objetivo principal es asegurarse que la información de los cambios llegue a todos los involucrados.

Algunas preguntas que podría responder (útiles para el responsable de configuración)

- ¿Cuál es el estado del ítem?
- ¿Un requerimiento de cambio ha sido aprobado o rechazado por el CCB?
- ¿Qué versión de ítem implementa un requerimiento de cambio aprobado (saber cuál es el componente que contiene la mejora)?
- ¿Cuál es la diferencia entre una versión y otra dada?

Plan de Gestión de Configuración ¡También se planifica!

¿Qué debería incluir el plan? EL PLAN DURA LO QUE DURA EL PROYECTO

- Reglas de nombrado de los CI
- Herramientas a utilizar para SCM
- Roles e integrantes del Comité
- Procedimiento formal de cambios
- Plantillas de formularios
- Procesos de Auditoría

Gestión de configuración en ambientes ágiles

En metodologías ágiles, la **gestión de configuración** cambia su enfoque: no se usa para controlar a los desarrolladores, sino para ayudar al equipo a coordinar el trabajo, seguir los cambios y mantener la integridad del producto.

Como los equipos ágiles son **autoorganizados**, algunas prácticas tradicionales se relativizan.

Por ejemplo, no suelen aplicarse auditorías formales ni comités rígidos de control de cambios.

La gestión de configuración se adapta al enfoque ágil porque permite responder al cambio sin perder calidad ni trazabilidad.

La diferencia principal es que el **Manifiesto Ágil** se enfoca en cómo trabajar dentro del proyecto, mientras que la **gestión de configuración** se enfoca en el producto y su calidad a lo largo del tiempo.

Características principales

- 1) La SCM responde a los cambios, no busca evitarlos.
- 2) Se automatiza todo lo posible para facilitar entregas rápidas y frecuentes.
- 3) Da retroalimentación continua sobre calidad, estabilidad e integridad.
- 4) Sus actividades están integradas dentro del trabajo del Sprint.
- 5) Es responsabilidad de todo el equipo.
- 6) Se apoya en prácticas continuas.

Continuous Integration

La **integración continua** consiste en que los desarrolladores integren su código varias veces al día en un repositorio compartido. Cada integración se verifica con pruebas automatizadas para detectar errores rápidamente.

Su objetivo es asegurar que el software pueda desplegarse en cualquier momento, es decir, que compile, pase pruebas y mantenga calidad.

Continuous Delivery

La **entrega continua** implica que el software se construya de manera tal que pueda ser liberado a producción en cualquier momento. Agrega automatización de pruebas de aceptación y despliegue automatizado.

Continuous Deployment

El **despliegue continuo** va un paso más allá: el software se despliega automáticamente en producción sin intervención humana, mediante pipelines que ejecutan pasos ordenados.

Canary Deployment

El **Canary Deployment** consiste en liberar una nueva versión primero a un pequeño grupo de usuarios. Si funciona bien y se valida el cambio, se extiende al resto.

Blue-Green Deployment

El **Blue-Green Deployment** usa dos entornos de producción similares: uno con la versión actual y otro con la nueva versión.

